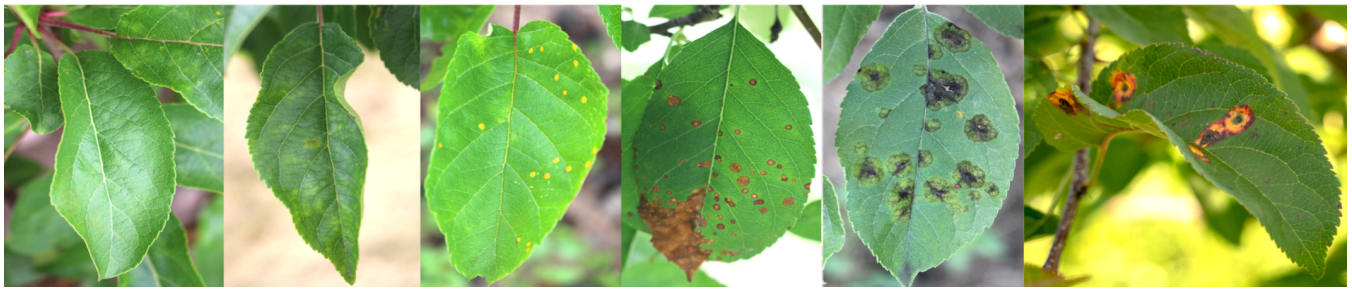
项目2-基于卷积神经网络的图像分类



如果有疑问，直接在文档对应的位置进行评论，这样让后面的同学可以看到。

一、基础知识

需要掌握的东西：数据维度是如何变换的？卷积核的计算过程？ResNet网络中的残差块？

不需要的掌握的东西：为什么？，我们便应用，解决问题即可。

参考：《鱼书》第7章以及8.1和8.2小节；刘二大人视频 p10 卷积神经网络（基础篇）

文件下载文档：项目2-可下载

1.1、 全连接层的局限性

全连接层的一个主要问题在于**忽略了输入数据的空间结构信息。**
当输入数据为图像时，其本质上是一个具有**高度、宽度和通道数**的三维数据（如图1-1所示）。例如，MNIST 数据集中每张图像的形状为 (1, 28, 28)，分别表示通道数、高度和宽度。然而，在输入全连接层之前，这样的三维数据通常需要被**展平成一维向量**，即将（1, 28, 28）转换为长度为 784 的向量。这个展平操作会破坏原有的数据结构，使得像素之间的空间关系无法被保留。

图像数据的三维结构中蕴含着重要的信息。例如，**空间上相邻的像素往往具有相似的数值特征**，不同颜色通道之间存在相关性，而距离较远的像素之间相关性较弱。这些信息对于图像理解任务至关重要。但全连接层会将所有输入神经元视为同等重要、彼此独立的元素进行处理，因而**难以有效利用图像的空间特性。**

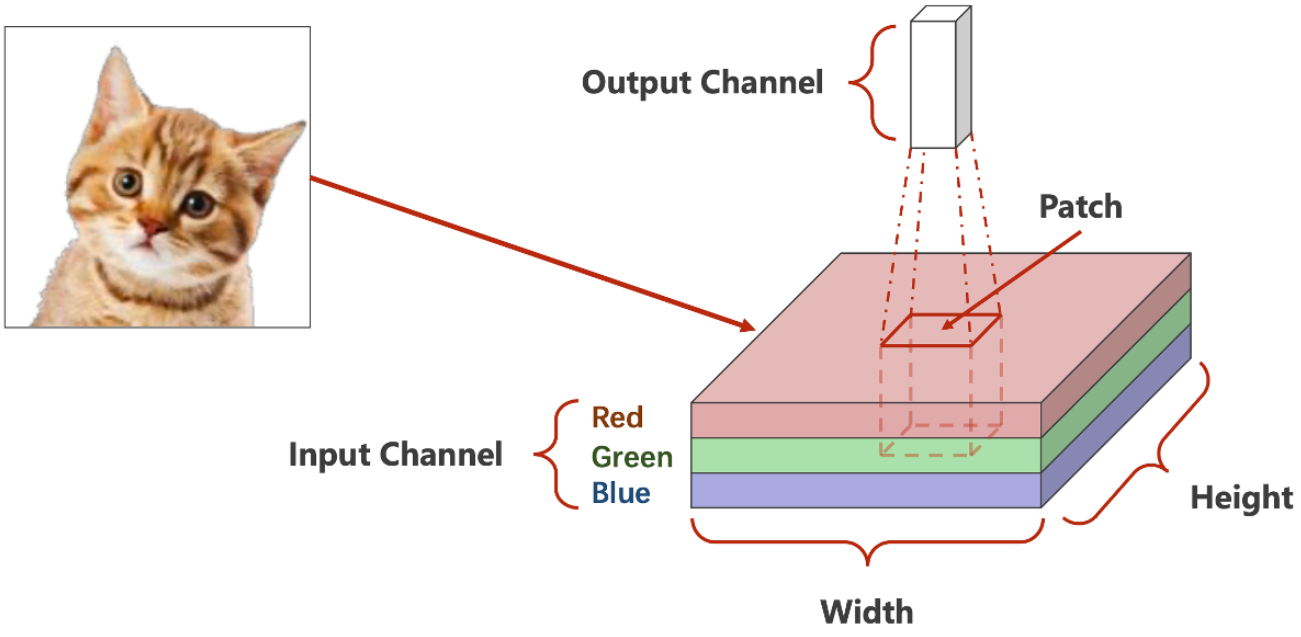


图1-1 图像的基本结构

1.2、卷积层

卷积层是卷积神经网络（Convolutional Neural Network, CNN）的核心组成部分，其主要作用是**从具有空间结构的数据中提取局部特征。**与全连接层不同，卷积层在计算过程中能够保持输入数据的空间结构，从而有效利用数据中蕴含的位置信息。

卷积层具有两个重要特性：

- **局部连接**：每个输出神经元只与输入数据中的一个局部区域相连接，而不是与所有输入神经元相连。
- **权值共享**：同一个卷积核在整个输入特征图上重复使用，从而减少了需要学习的参数数量。

由于这两个特性，卷积层在参数规模和计算效率上都优于全连接层，同时也更适合处理图像等具有空间结构的数据。

1.2.1 单输入通道卷积运算示意

图1-2中展示了二维卷积运算在输入数据上的具体计算过程。这里假设输入为**单通道二维数据**，卷积核大小为 (3×3)，步幅（stride）为 1，且不使用填充（padding）。

- **第一次卷积计算**

第一张图中，卷积核位于输入特征图的左上角。此时，卷积核覆盖输入数据的一个 (3×3) 局部区域。卷积运算的过程为：

1. 将卷积核与对应的输入区域按元素相乘；
2. 对所有乘积结果求和；
3. 得到一个标量作为输出特征图中的一个元素。

该位置的计算结果为： $3 \cdot 1 + 4 \cdot 2 + 6 \cdot 3 + \cdots + 7 \cdot 9 = 211$。因此，输出特征图左上角的值为 **211**。

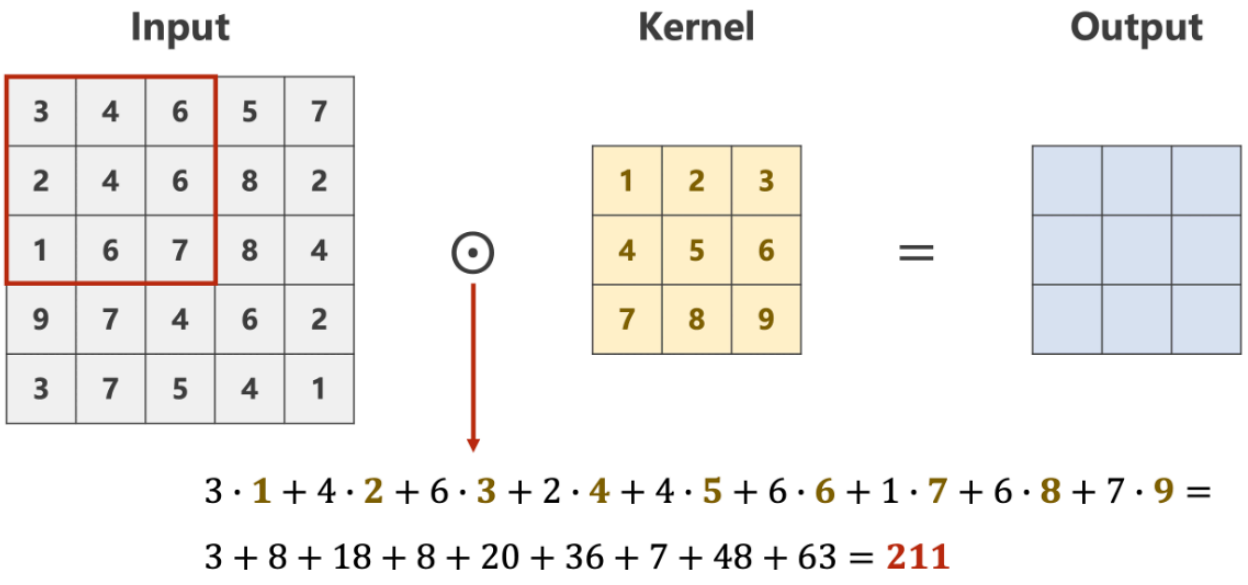


图1-2 单输入通道卷积计算过程示意图

- **卷积核的水平移动**

在图1-3中，卷积核向**右移动一个位置**（步幅为 1），并对新的 (3×3) 输入区域执行同样的运算。

在该位置，输入区域发生变化，但**卷积核的权重保持不变**。通过相同的逐元素乘加操作，得到新的输出值： $4 \cdot 1 + 6 \cdot 2 + 5 \cdot 3 + \cdots + 8 \cdot 9 = 295$。该结果填入输出特征图中对应的位置，即第一行第二列。

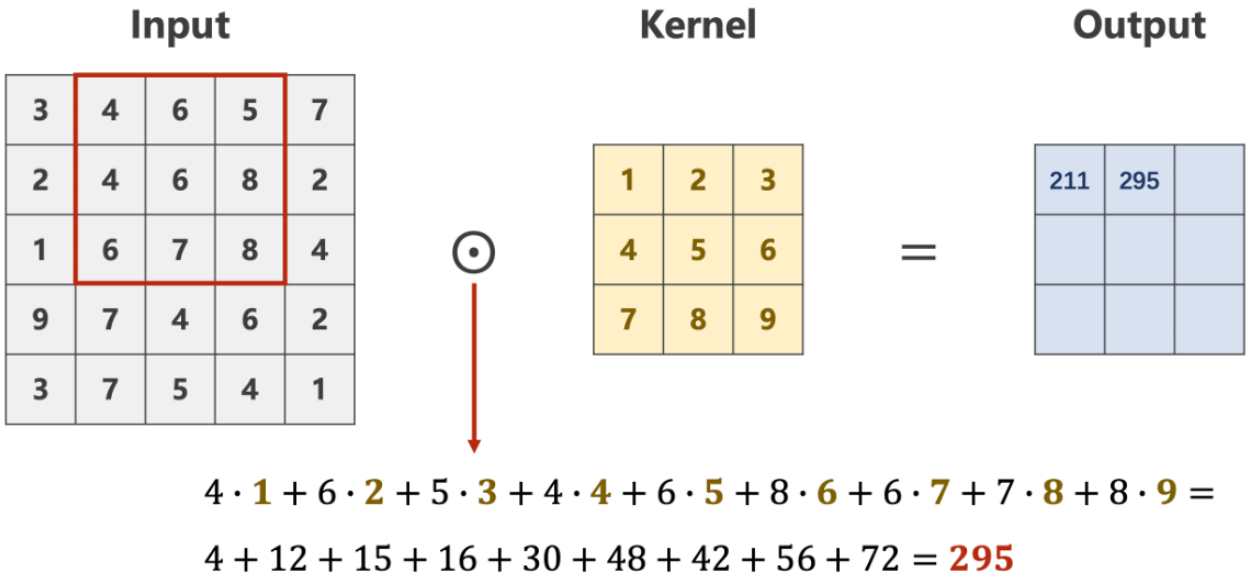


图1-3 单输入通道卷积计算过程示意图

- **遍历输入并生成输出特征图**

图1-4展示了卷积核在输入特征图上**逐行、逐列滑动**的过程。每一次滑动，都会对当前覆盖的局部区域执行相同的卷积计算，并将结果依次填入输出特征图。

当卷积核无法完整覆盖输入区域时（即超出边界），计算终止。最终，输入大小为 (5×5)，卷积核大小为 (3×3) 时，输出特征图大小为： $(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$。输出特征图中的每一个元素，都表示输入数据在对应局部区域内与该卷积核的**响应强度**。

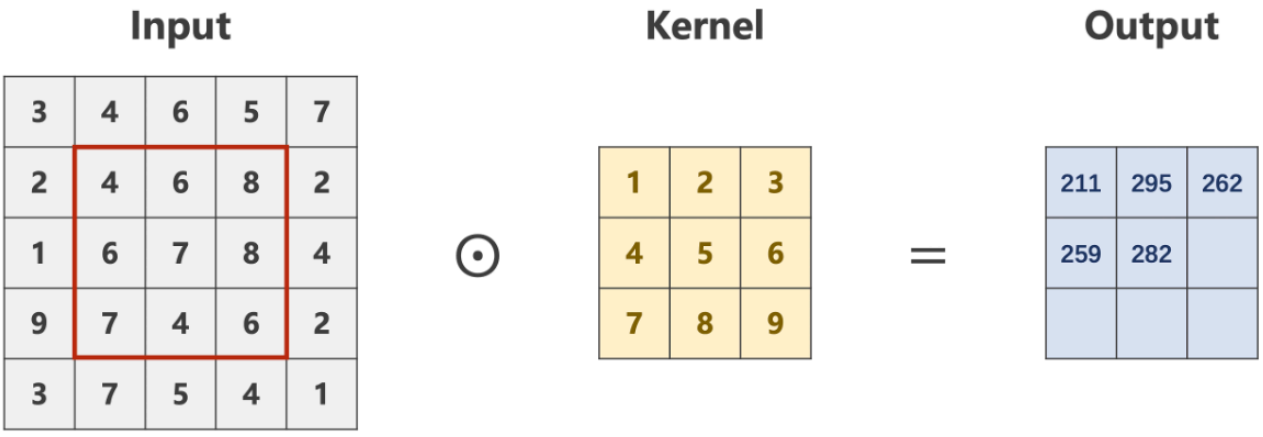


图1-4 单输入通道卷积计算过程示意图

- **小结及代码示例**
  - 卷积运算通过**局部感受野**提取输入数据中的局部特征；
  - 同一卷积核在不同位置重复使用，体现了**权值共享**的思想；

这种运算方式使得卷积层能够高效地提取空间结构信息，是卷积神经网络处理图像数据的基础。

> 对应文件：conv2d_input_1.py

```
import torch
input = [3,4,6,5,7,
        2,4,6,8,2,
        1,6,7,8,4,
        9,7,4,6,2,
        3,7,5,4,1]
input = torch.Tensor(input).view(1, 1, 5, 5)
conv_layer = torch.nn.Conv2d(1, 1, kernel_size=3, bias=False)
kernel = torch.Tensor([1,2,3,4,5,6,7,8,9]).view(1, 1, 3, 3)
conv_layer.weight.data = kernel.data
print(conv_layer.weight.data )
output = conv_layer(input)
print(output)

'''
tensor([[[[1., 2., 3.],
          [4., 5., 6.],
          [7., 8., 9.]]]])
tensor([[[[211., 295., 262.],
          [259., 282., 214.],
          [251., 253., 169.]]]], grad_fn=<ConvolutionBackward0>)
'''
```

1.2.2 三输入通道的卷积运算

当输入数据具有多个通道时（例如彩色图像的 RGB 三通道），卷积层的运算方式与单通道情形在本质上是相同的，但**在通道方向上需要进行加权求和。**

图1-5中，输入由 **3 个通道**组成，每个通道都是一个大小为 (5×5) 的二维特征图。对应地，一个卷积核不再是单个二维矩阵，而是由 **3 个 (** 3×3**) 的权重矩阵**叠加而成，即：

$$\text{卷积核形状} = (3,3,3)$$

其中，**卷积核在通道方向上的深度必须与输入特征图的通道数一致。**

- **通道内卷积与通道间求和**

对每一个输入通道，分别执行二维卷积运算，得到 3 张中间特征图（图1-5中以不同颜色表示）。随后，将这 3 张特征图在**对应位置进行逐元素相加**，从而得到一张最终的输出特征图。这一过程可以概括为：

1. 对每个通道独立进行卷积；
2. 将各通道的卷积结果求和；
3. 得到单通道的输出特征图。

因此，**一个卷积核只能生成一张输出特征图。**

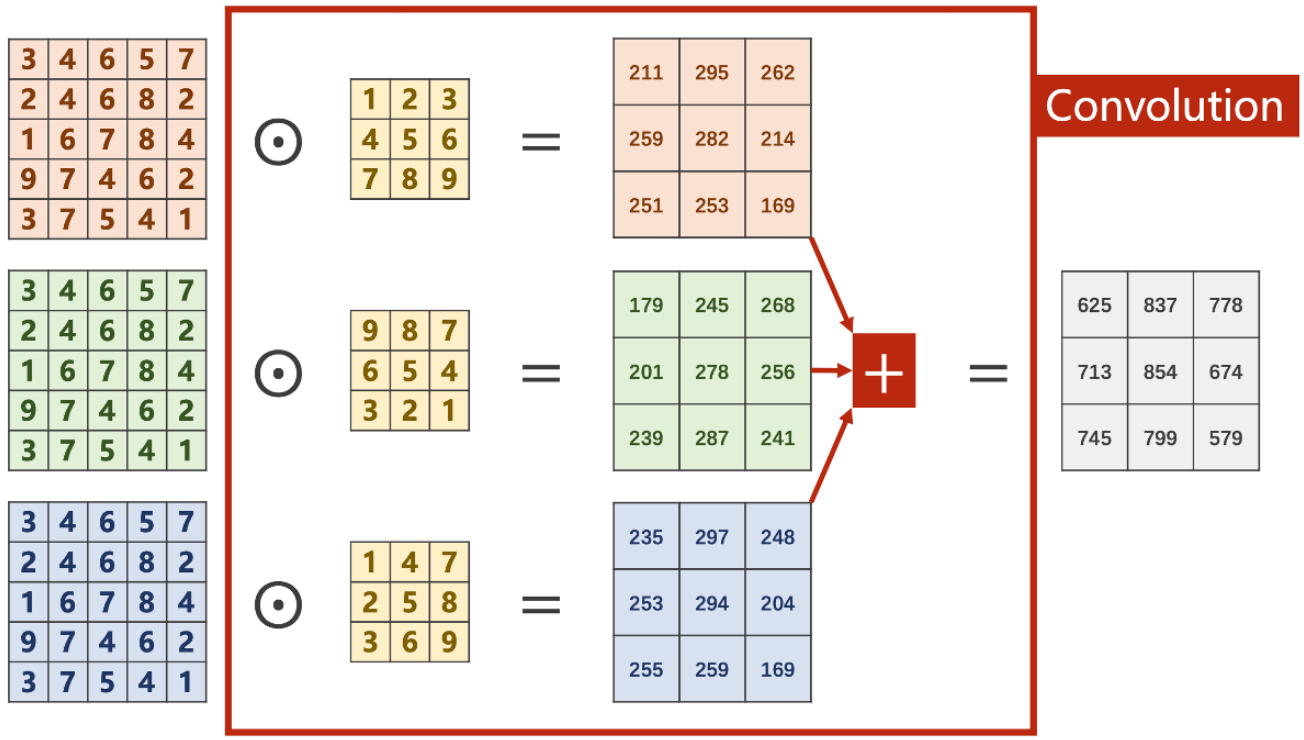


图1-5 三输入通道卷积计算通过示意图

- **输出特征图的尺寸变化**

图1-6从整体张量形状的角度，对上述过程进行了总结说明。

- 输入特征图形状为： $(\text{channel}=3,\text{height}=5,\text{width}=5)$
- 卷积核大小为： $3 \times 3 \times 3$
  （深度为 3，与输入通道数一致）
- 在步幅为 1、无填充（padding）的情况下，输出特征图的空间尺寸为：
  $(5 - 3 + 1) \times (5 - 3 + 1) = 1\times 3 \times 3$
- 最终输出特征图形状为：$(\text{channel}=1,\text{height}=3, \text{width}=3)$

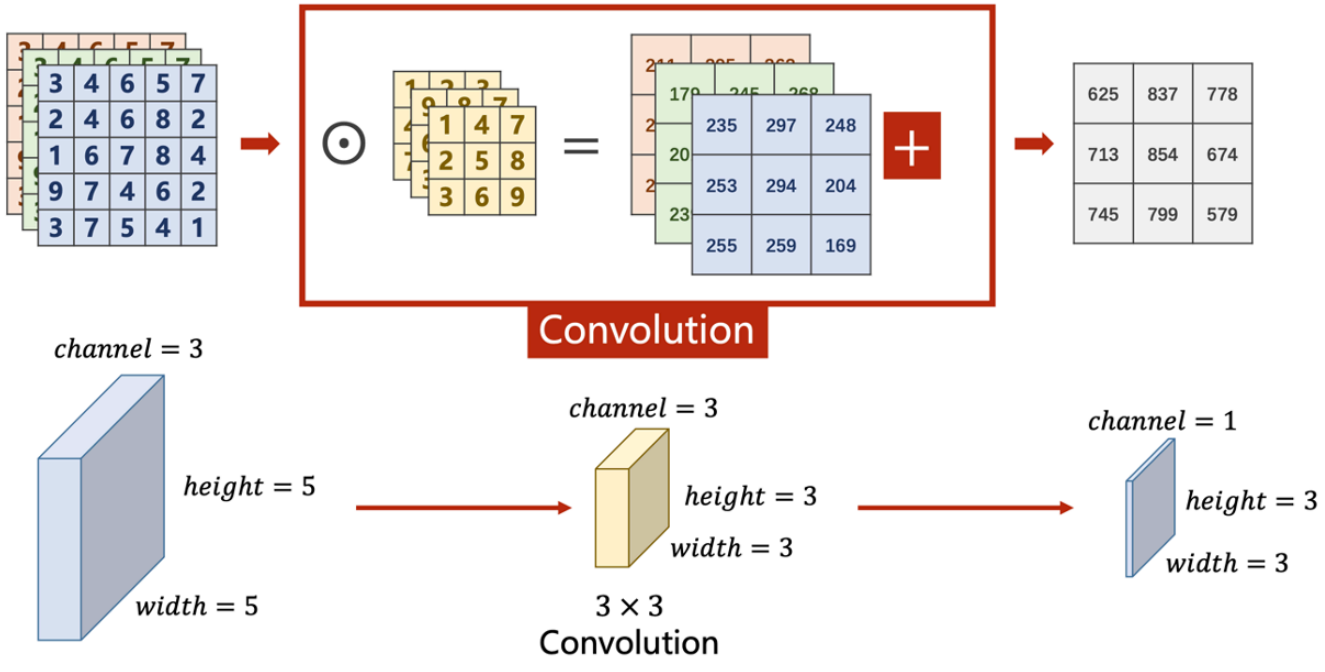


图1-6 三输入通道卷积计算过程示意图

- 小结及代码示例
    - 多通道卷积中，**卷积核在通道方向上必须具有与输入相同的深度**；
    - 每个通道独立进行卷积运算，随后在通道方向上进行求和；
    - 一个卷积核对应一张输出特征图；
    - 若使用多个卷积核，则可以得到多张输出特征图，从而形成新的多通道输出。

> 对应文件：conv2d_input_3.py

```
import torch
import torch.nn as nn

# ====================== 1. 定义3通道输入张量 ======================
# 输入形状说明: [batch_size, in_channels, height, width] -> [1, 3, 5, 5]
input_tensor = torch.tensor(
    [
        # 输入通道1
        [
            [3, 4, 6, 5, 7],
            [2, 4, 6, 8, 2],
            [1, 6, 7, 8, 4],
            [9, 7, 4, 6, 2],
            [3, 7, 5, 4, 1]
        ],
        # 输入通道2
        [
            [3, 4, 6, 5, 7],
            [2, 4, 6, 8, 2],
            [1, 6, 7, 8, 4],
            [9, 7, 4, 6, 2],
            [3, 7, 5, 4, 1]
        ],
        # 输入通道3
        [
            [3, 4, 6, 5, 7],
            [2, 4, 6, 8, 2],
            [1, 6, 7, 8, 4],
            [9, 7, 4, 6, 2],
            [3, 7, 5, 4, 1]
        ]
    ],
    dtype=torch.float32
).unsqueeze(0)  # 增加batch维度，适配Conv2d输入要求

# ====================== 2. 定义卷积层（禁用默认权重初始化） ======================
# 卷积层参数说明:
# - in_channels=3: 匹配输入的3通道
# - out_channels=1: 仅输出1个通道
# - kernel_size=3: 3x3卷积核
# - stride=1: 步幅为1
# - padding=0: 无填充
# - bias=False: 不使用偏置项（与原始计算逻辑一致）
```

```
conv_layer = nn.Conv2d(
    in_channels=3,
    out_channels=1,
    kernel_size=3,
    stride=1,
    padding=0,
    bias=False
)

# ====================== 3. 手动赋值卷积核权重 ======================
# 自定义权重形状: [out_channels, in_channels, kernel_h, kernel_w] -> [1, 3, 3, 3]
custom_weight = torch.tensor(
    [
        [
            # 输入通道1对应的3x3卷积核
            [[1, 2, 3], [4, 5, 6], [7, 8, 9]],
            # 输入通道2对应的3x3卷积核
            [[9, 8, 7], [6, 5, 4], [3, 2, 1]],
            # 输入通道3对应的3x3卷积核
            [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
        ]
    ],
    dtype=torch.float32
)

# 将自定义权重赋值给卷积层（requires_grad=False：权重固定，不参与训练）
conv_layer.weight = nn.Parameter(custom_weight, requires_grad=False)

# ====================== 4. 执行卷积计算并输出结果 ======================
# 禁用梯度计算：推理阶段无需计算梯度，提升效率且节省内存
with torch.no_grad():
    conv_output = conv_layer(input_tensor)

# 打印最终卷积输出结果
print("卷积计算输出结果：")
print(conv_output)
```

1.2.3 多通道输入下的卷积核结构

图1-7展示了**输入通道数为 (n)** 时，单个卷积核的结构及其对输出的影响。

设输入特征图的形状为：

$$(n,H, W)$$

其中 (n) 表示通道数，(H) 和 (W) 分别表示特征图的高度和宽度。

当使用大小为 (3×3) 的卷积运算时，卷积核在**空间维度上为 (** 3×3**)**，并且在**通道维度上的深度必须与输入通道数一致**。因此，单个卷积核的张量形状为： $(n,3,3)$

卷积运算时，该卷积核会同时作用于输入的所有通道，在每个通道上分别进行二维卷积，并在通道方向上将结果相加，从而生成一张输出特征图。

在不使用填充、步幅为 1 的情况下，若输入空间尺寸为 (5×5)，则输出特征图的空间尺寸为： $(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$

最终，该卷积核产生的输出特征图形状为： $(1, 3, 3)$

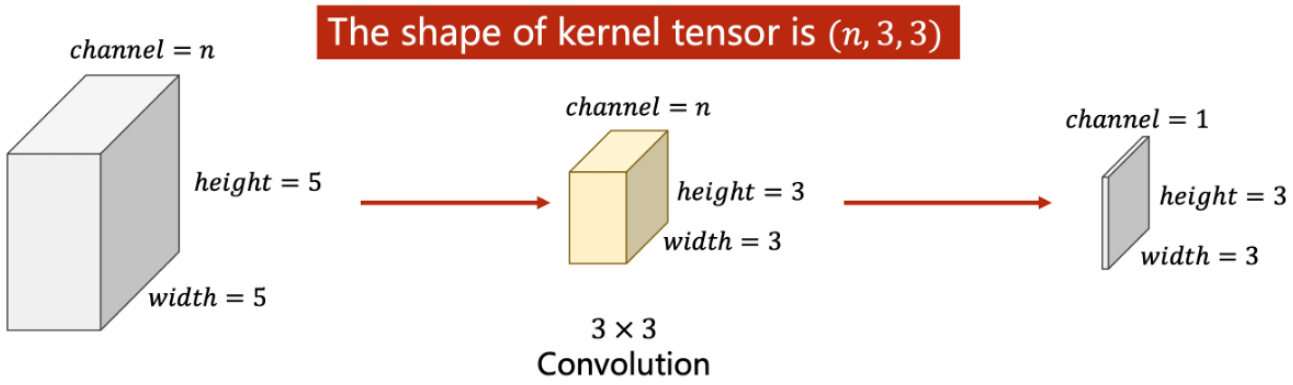


图1-7 多输入通道卷积计算过程示意图

1.2.4 多滤波器与多输出通道

图1-8和图1-9进一步展示了**多个卷积核（滤波器）同时作用于同一输入特征图**的情形。设输入特征图的形状为：$(n,\text{width}_{\text{in}},\text{height}_{\text{in}})$。

若在该卷积层中使用了 (m) 个卷积核：

- 每个卷积核的形状均为 (n, k, k)；
- 每个卷积核都会独立生成一张输出特征图。

因此，经过卷积运算后，可以得到 (m) 张输出特征图，分别记为： $\text{Feature map}_1, \text{Feature map}_2, \ldots, \text{Feature map}_m$

这些特征图在通道维度上进行堆叠（concatenate），形成最终的输出张量，其形状为： $(m, \text{width}_{\text{out}},\text{height}_{\text{out}})$

其中，输出的通道数 **由卷积核（滤波器）的个数决定。**

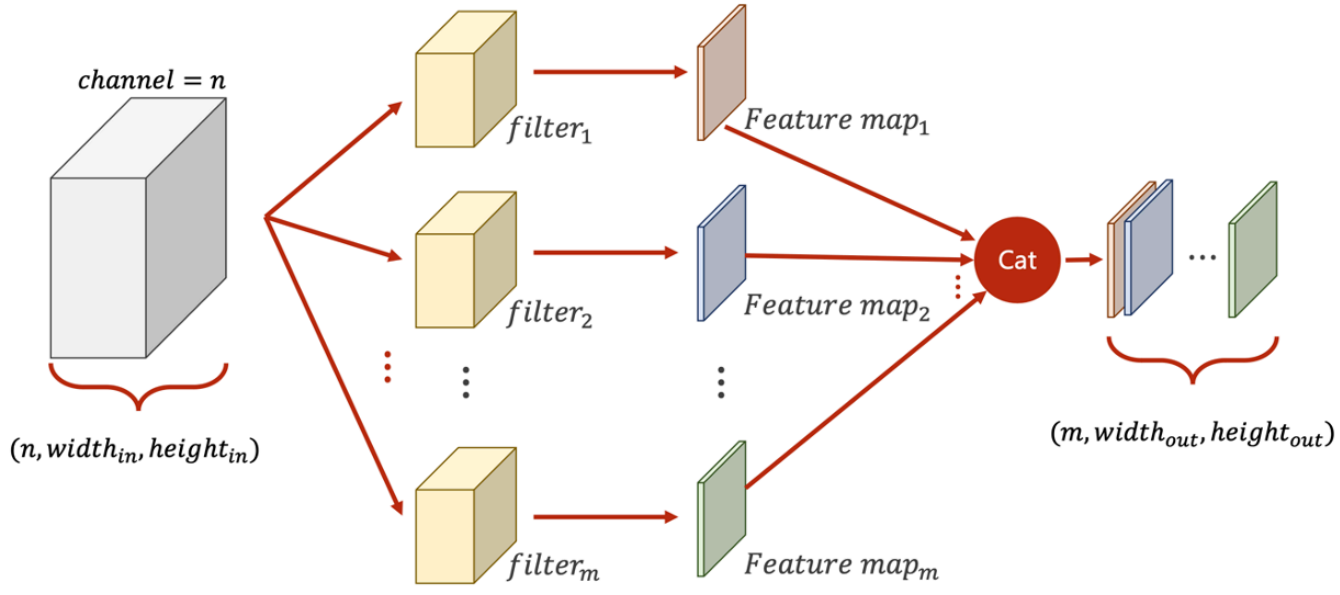


图1-8 多输入通道和多输出通道卷积计算过程示意图

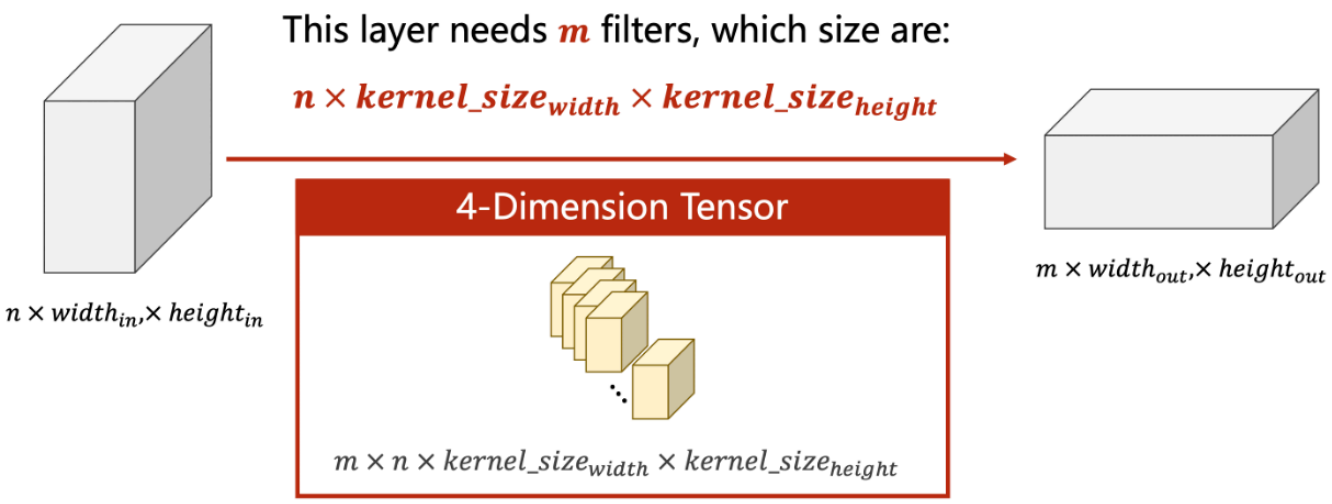


图1-9 多输入通道和多输出通道卷积计算过程示意图

- 小结及代码示例
  - 输入为 (n) 通道时，**单个卷积核的深度必须为 (n)**；
  - 一个卷积核只能生成 **一张输出特征图**；
  - 使用多个卷积核可以得到多通道输出；
  - 卷积层的输出通道数等于卷积核的数量。

> 对应文件：conv2d_input_n.py

```
import torch
in_channels, out_channels= 5, 10
width, height = 100, 100
kernel_size = 3
batch_size = 1

input = torch.randn(batch_size,
                    in_channels,
                    width,
                    height)

conv_layer = torch.nn.Conv2d(in_channels,
                    out_channels,
                    kernel_size=kernel_size)

output = conv_layer(input)
print(input.shape)
print(output.shape)
print(conv_layer.weight.shape)
```

1.2.5 卷积层-填充

- padding = 1

图1-10中展示了在卷积运算中引入**零填充（zero padding）** 的情形。与不使用填充的卷积相比，填充操作通过在输入特征图的边界处补零，使卷积核能够覆盖到原始输入的边缘区域。

- 填充后的输入特征图

原始输入为一个二维特征图。在其上下左右各填充一圈值为 0 的元素后，输入特征图的空间尺寸增大。图1-10中红色的“0”表示填充区域，这些数值并非来自原始数据，而是人为添加的边界信息。通过填充，原始输入中的边界像素在卷积计算中能够与卷积核完整地参与运算，而不会被忽略。

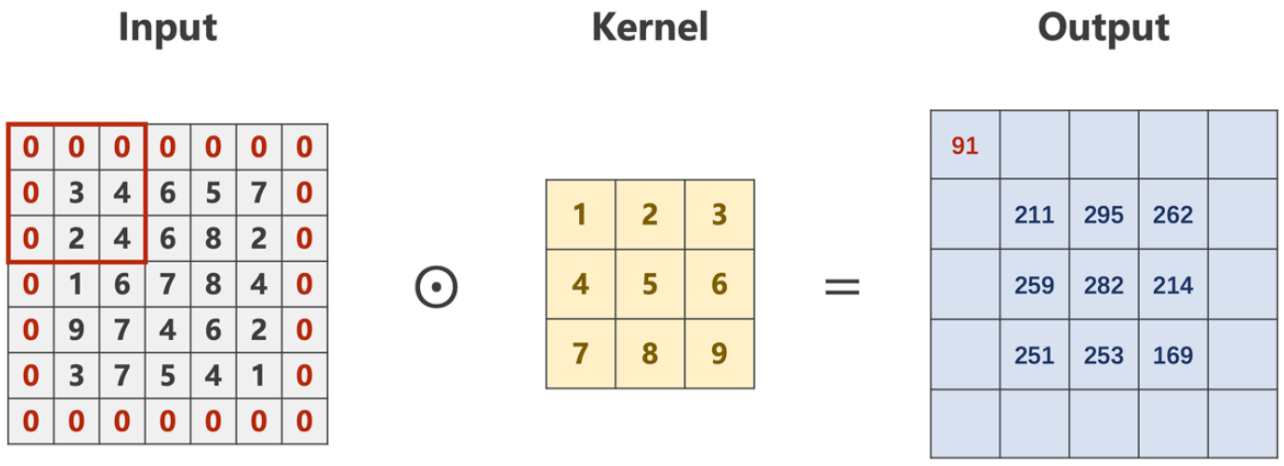


图1-10 卷积加入padding参数后的计算过程示意图

- 填充的意义
  - **保留空间尺寸**，避免特征图在多层卷积后迅速缩小；
  - **强化边界信息的利用**，使边缘像素在特征提取中具有与内部像素相近的重要性；
  - 便于构建深层网络，保持特征图尺寸的可控性。

> 对应文件：conv2d_input_1_padding.py

```
import torch
input = [3,4,6,5,7,
         2,4,6,8,2,
         1,6,7,8,4,
         9,7,4,6,2,
         3,7,5,4,1]
input = torch.Tensor(input).view(1, 1, 5, 5)

conv_layer = torch.nn.Conv2d(1, 1, kernel_size=3,  padding=1, bias=False)

kernel = torch.Tensor([1,2,3,4,5,6,7,8,9]).view(1, 1, 3, 3)

conv_layer.weight.data = kernel.data

print(conv_layer.weight.data )
output = conv_layer(input)
```

```
print(output)

'''
tensor([[[[1., 2., 3.],
          [4., 5., 6.],
          [7., 8., 9.]]]])
tensor([[[[ 91., 168., 224., 215., 127.],
          [114., 211., 295., 262., 149.],
          [192., 259., 282., 214., 122.],
          [194., 251., 253., 169.,  86.],
          [ 96., 112., 110.,  68.,  31.]]]], grad_fn=<ConvolutionBackward0>)
'''
```

1.2.6 卷积层-步长

- 步长=2

步长（stride）是指在卷积运算中，**卷积核在输入特征图上每次滑动的像素数。**在二维卷积中，步长通常在高度方向和宽度方向上取相同的值，记为 (S)。

- 步长的作用

如图1-11，在进行卷积运算时，卷积核从输入特征图的左上角开始，按固定的步长在水平方向和垂直方向上依次移动，并在每个位置执行一次卷积计算。步长的大小直接决定了**卷积核采样输入特征图的密度。**

- 当步长较小时，卷积核滑动得更加密集，能够更细致地提取局部特征；
- 当步长较大时，卷积核跳跃式移动，会降低特征图的空间分辨率。

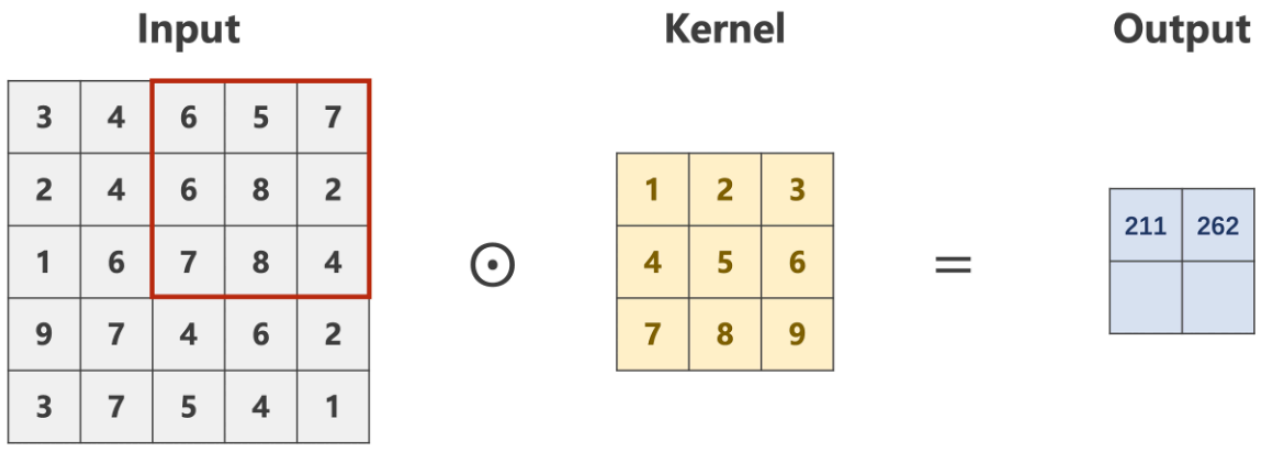


图1-11 卷积引入步长参数后的计算过程示意图

> 对应文件：conv2d_input_1_stride.py

```
import torch
input = [3,4,6,5,7,
         2,4,6,8,2,
         1,6,7,8,4,
         9,7,4,6,2,
```

```
        3,7,5,4,1]
input = torch.Tensor(input).view(1, 1, 5, 5)

conv_layer = torch.nn.Conv2d(1, 1, kernel_size=3,  stride=2,  bias=False)

kernel = torch.Tensor([1,2,3,4,5,6,7,8,9]).view(1, 1, 3, 3)

conv_layer.weight.data = kernel.data

print(conv_layer.weight.data )
output = conv_layer(input)
print(output)

'''
tensor([[[[1., 2., 3.],
          [4., 5., 6.],
          [7., 8., 9.]]]])
tensor([[[[211., 262.],
          [251., 169.]]]], grad_fn=<ConvolutionBackward0>)
'''
```

1.2.7 torch代码实践

以图形作为输入来做卷积，下面代码演示了使用 PyTorch 对单张图像进行卷积处理的基本流程。程序首先通过 PIL 读取输入图像，并利用 torchvision.transforms 将其转换为符合神经网络输入要求的张量格式；随后构建一个二维卷积层（Conv2d），对图像进行卷积运算以提取局部特征；最后将卷积输出的张量重新转换为图像格式并保存到磁盘。

> 对应文件：conv2d_image.py

```
import torch
import torchvision.transforms as transforms
from PIL import Image

# 定义图像预处理步骤，这里将图像转换为张量，可根据需求添加图像大小调整等操作
transform = transforms.Compose([
    # transforms.Resize((256, 256)),  # 如果需要可以调整图像大小
    transforms.ToTensor()  # 转换为张量
])

if __name__ == '__main__':
    image_path = "./input_image.jpg"
    # 加载图像并转换为RGB模式
    image = Image.open(image_path).convert("RGB")

    # 将图像转换为张量并添加批次维度，以便后续输入到神经网络层
    input_tensor = transform(image).unsqueeze(0)

    # 定义卷积层参数
    in_channels, out_channels = 3, 3
    kernel_size = 10
    # 创建卷积层
```

```
    conv_layer = torch.nn.Conv2d(in_channels, out_channels, kernel_size=kernel_size)
    # 使用卷积层处理图像
    output = conv_layer(input_tensor)

    # 将处理后的张量转换回PIL图像格式
    output_image = transforms.ToPILImage()(output.squeeze(0))

    # 保存处理后的图像
    output_image.save("./output_image.jpg")
```

1.3、池化层

1.3.1 基本介绍

池化层是卷积神经网络中常用的一种下采样结构，其主要作用是**对特征图进行空间压缩，同时保留重要特征信息。**
池化层通常位于卷积层之后，用于减少特征图的空间尺寸，从而降低计算量并提高模型的鲁棒性。

- 池化运算的基本思想

池化层通过在输入特征图上划分若干局部区域，并对每个区域内的数值进行某种**汇聚（pooling）操作**，从而得到输出特征图。与卷积层不同，池化层**不包含可学习参数**，其运算方式是固定的。

- 常见的池化方式
    1. **最大池化（Max Pooling）**
       在局部区域内取最大值作为输出，如图1-12所示。
       最大池化能够保留最显著的特征响应，因此在实际应用中最为常见。
    2. **平均池化（Average Pooling）**
       在局部区域内取平均值作为输出。
       平均池化更加关注整体信息，但对显著特征的保留能力较弱。

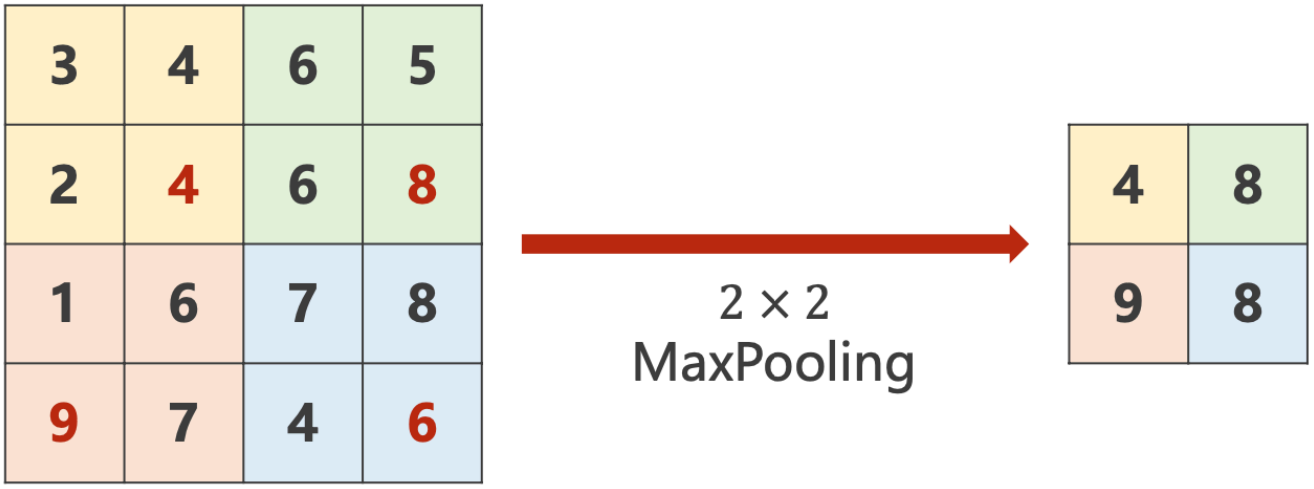


图1-12 最大池化计算过程示意图

> 对应文件：maxpool2d.py

```
import torch
input = [3,4,6,5,
        2,4,6,8,
        1,6,7,8,
        9,7,4,6,
        ]
input = torch.Tensor(input).view(1, 1, 4, 4)
maxpooling_layer = torch.nn.MaxPool2d(kernel_size=2)
output = maxpooling_layer(input)
print(output)
```

- 池化窗口与步长

池化运算通常使用固定大小的窗口（如 (2×2)），并按照一定的步长在特征图上滑动。
常见设置为 **池化窗口大小等于步长**，例如 (2×2) 池化配合步长为 2，从而使特征图的高度和宽度各缩小一半。

- 池化层的作用
    - **降低特征图的空间分辨率**，减少计算量；
    - **抑制局部位置变化的影响**，提高模型对平移等微小变化的鲁棒性；

1.3.2 torch代码实践

本示例代码演示了使用 PyTorch 对单张图像进行最大池化（Max Pooling）操作的基本流程。程序首先读取输入图像并将其转换为张量格式，同时添加批次维度以满足神经网络输入要求；随后构建一个二维最大池化层，通过在局部区域内取最大值的方式对图像进行下采样，从而减小特征图尺寸并增强对显著特征的鲁棒性；最后将池化后的结果转换回图像格式并保存。

> 对应文件：pool2d_image.py

```
import torch
import torchvision.transforms as transforms
from PIL import Image

# 定义图像预处理步骤，这里将图像转换为张量，可根据需求添加图像大小调整等操作
transform = transforms.Compose([
    # transforms.Resize((256, 256)),  # 如果需要可以调整图像大小
    transforms.ToTensor()  # 转换为张量
])

# 指定输入图像路径，这里需要替换为实际的图像路径
image_path = "./input_image.jpg"
# 加载图像并转换为RGB模式
image = Image.open(image_path).convert("RGB")

# 将图像转换为张量并添加批次维度，以便后续输入到神经网络层
input_tensor = transform(image).unsqueeze(0)

# 定义最大池化层参数
kernel_size = 5
# 创建最大池化层
```

```
maxpooling_layer = torch.nn.MaxPool2d(kernel_size=kernel_size)
# 使用最大池化层处理图像
output = maxpooling_layer(input_tensor)

# 将处理后的张量转换回PIL图像格式
output_image = transforms.ToPILImage()(output.squeeze(0))

# 保存处理后的图像
output_image.save("./output_image_maxpool.jpg")
```

1.4、搭建一个卷积网络

> 能够看懂维度的变换即可，不需要深入思考为什么这么设计。

图1-13展示了一个典型的卷积神经网络（CNN）结构及其特征尺寸变化过程。网络以大小为 (28×28) 的单通道图像作为输入，首先通过一个 (5×5) 的卷积层将通道数从 1 扩展到 10，提取初级局部特征；随后经过 (2×2) 的池化层对特征图进行下采样，减少空间尺寸并增强特征鲁棒性。接着，网络再次堆叠卷积层和池化层，将特征通道数提升至 20，并进一步压缩空间分辨率。最后，将得到的高层特征展开并送入全连接层，输出长度为 10 的向量，用于完成分类任务。

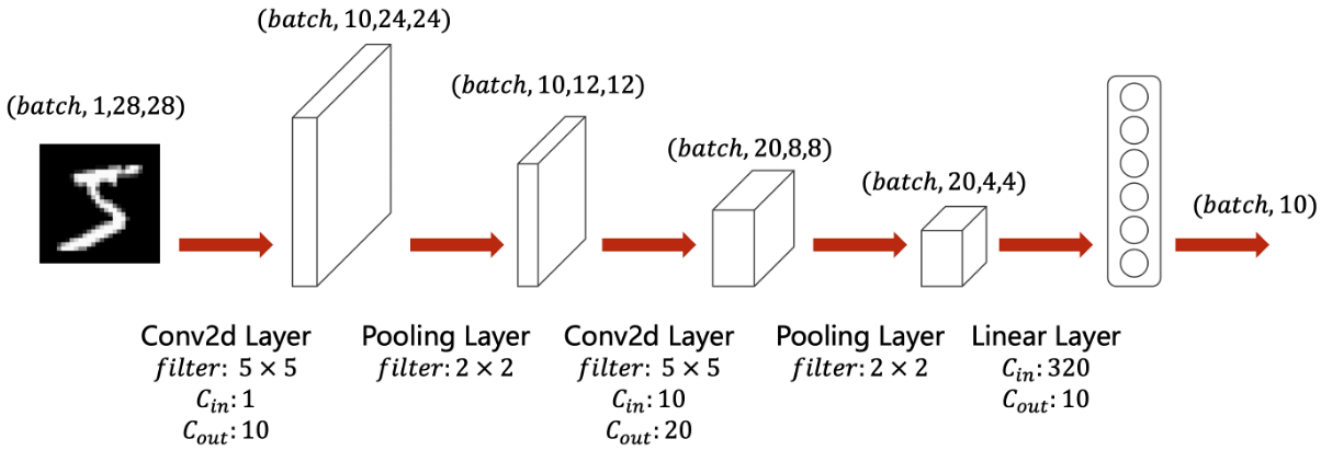
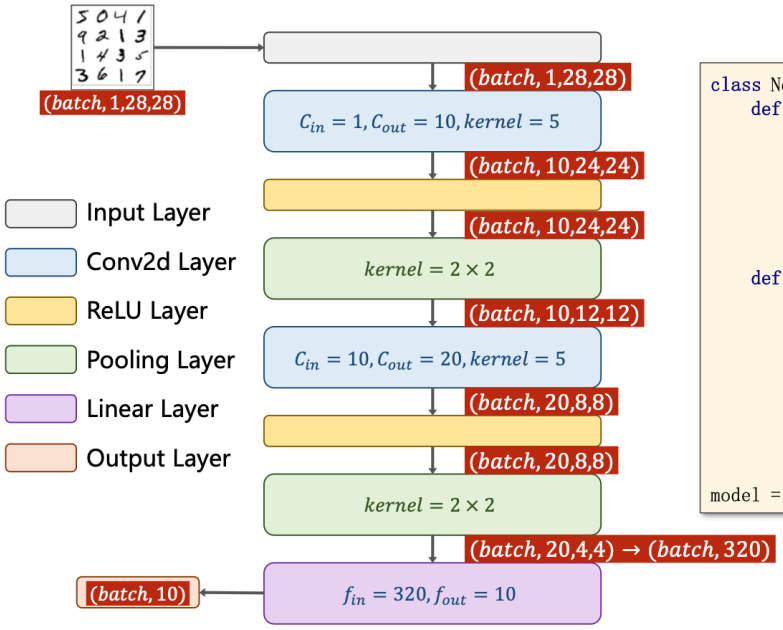


图1-13 简单的卷积网络网络结构图

图1-14为对应网络结构的torch代码。




```
class Net(torch.nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = torch.nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = torch.nn.Conv2d(10, 20, kernel_size=5)
        self.pooling = torch.nn.MaxPool2d(2)
        self.fc = torch.nn.Linear(320, 10)

    def forward(self, x):
        # Flatten data from (n, 1, 28, 28) to (n, 784)
        batch_size = x.size(0)
        x = F.relu(self.pooling(self.conv1(x)))
        x = F.relu(self.pooling(self.conv2(x)))
        x = x.view(batch_size, -1) # flatten
        x = self.fc(x)
        return x

model = Net()
```

图1-14 简单的卷积网络和torch代码结构图

> 对应文件：sample_net.py

```python
# 导入必要的Python库和PyTorch相关模块
import os                          # 用于文件路径处理、目录创建等操作系统交互
import torch                       # PyTorch核心库，提供张量计算和神经网络构建能力
import torch.nn as nn              # PyTorch神经网络模块，包含卷积层、全连接层等基础组件
import torch.nn.functional as F    # PyTorch函数式接口，提供ReLU、池化等无参数操作
from torchvision import transforms  # 提供图片预处理（如转张量、缩放）的工具
from PIL import Image              # Python Imaging Library，用于读取和保存图片

# ==========================
# 1. 定义MNIST手写数字识别网络（适配单通道28x28灰度图）
# ==========================
class Net(nn.Module):
    """
    自定义MNIST分类网络结构：
    输入：单通道28x28灰度图 -> 输出：10维数字分类概率（0-9）
    网络结构：Conv1 → MaxPool → ReLU → Conv2 → MaxPool → ReLU → Flatten → FC → 输出
    """
    def __init__(self):
        # 继承父类nn.Module的初始化方法，必须调用
        super(Net, self).__init__()

        # 第一层卷积：输入通道1（灰度图），输出通道10，卷积核5x5，步幅默认1，无填充
        # 输入形状(1,1,28,28) → 卷积后(1,10,24,24)（28-5+1=24）
        self.conv1 = nn.Conv2d(
            in_channels=1,     # 输入通道数：MNIST是单通道灰度图
            out_channels=10,   # 输出通道数：10个卷积核，提取10种特征
            kernel_size=5      # 卷积核尺寸：5x5
        )

        # 第二层卷积：输入通道10（对应conv1的输出），输出通道20，卷积核5x5
        # 输入形状(1,10,12,12)（conv1池化后）→ 卷积后(1,20,8,8)（12-5+1=8）
        self.conv2 = nn.Conv2d(
```

```
            in_channels=10,    # 输入通道数匹配conv1的输出通道
            out_channels=20,   # 输出通道数：20个卷积核，提取更复杂的特征
            kernel_size=5      # 卷积核尺寸：5x5
        )

        # 全连接层：输入维度320，输出维度10（对应0-9数字分类）
        # 320的计算逻辑：conv2池化后形状(1,20,4,4) → 20*4*4=320
        self.fc = nn.Linear(
            in_features=320,   # 输入特征数：展平后的特征维度
            out_features=10    # 输出类别数：MNIST共10个数字类别
        )

        # 最大池化层：池化核2x2，步幅默认等于核尺寸（即2），无填充
        # 作用：降维、保留关键特征、防止过拟合
        self.pooling = nn.MaxPool2d(kernel_size=2)

    def forward(self, x):
        """
        定义网络前向传播逻辑（核心！PyTorch会自动构建反向传播）
        参数x：输入张量，形状为(batch_size, channels, height, width)
        返回值：网络输出张量，形状为(batch_size, 10)
        """
        # 第一层：卷积 → 池化 → ReLU激活
        # x: (1,1,28,28) → conv1 → (1,10,24,24) → pooling → (1,10,12,12) → ReLU →
(1,10,12,12)
        x = F.relu(self.pooling(self.conv1(x)))

        # 第二层：卷积 → 池化 → ReLU激活
        # x: (1,10,12,12) → conv2 → (1,20,8,8) → pooling → (1,20,4,4) → ReLU → (1,20,4,4)
        x = F.relu(self.pooling(self.conv2(x)))

        # 展平操作：将4维张量转为2维（batch_size, 特征数）
        # x: (1,20,4,4) → view → (1, 320)
        # x.size(0)是batch_size（这里为1），-1表示自动计算剩余维度
        x = x.view(x.size(0), -1)

        # 全连接层：展平后的特征 → 10维分类输出
        # x: (1,320) → fc → (1,10)
        x = self.fc(x)

        # 返回最终输出（注：此处未加Softmax，输出是logits，需后续处理得到概率）
        return x

# =========================
# 2. 初始化模型 + 设置推理模式
# =========================
# 实例化自定义网络
model = Net()
# 设置模型为评估模式（eval）：禁用Dropout、BatchNorm等训练相关层的行为
# 推理/预测时必须调用，否则结果会受训练模式影响
model.eval()

# =========================
```

```
# 3. 加载并预处理MNIST图片
# ==========================
# 本地MNIST图片路径（需替换为你实际的图片路径）
img_path = "mnist_sample/mnist_2_label_4.png"
# 使用PIL.Image打开图片（MNIST是单通道灰度图，无需convert("RGB")）
img = Image.open(img_path)

# 定义图片预处理流水线
transform = transforms.Compose([
    # 注释掉的Resize：MNIST原始尺寸是28x28，无需缩放到256x256
    # transforms.Resize((256, 256)),
    # 将PIL Image转为PyTorch张量：
    # 输入：PIL灰度图（0-255）→ 输出：张量（0.0-1.0），形状为(1,28,28)
    transforms.ToTensor()
])

# 对图片执行预处理，并增加batch维度（模型要求输入必须有batch维度）
# x: (1,28,28) → unsqueeze(0) → (1,1,28,28)
# 注：原注释标注的(1,3,256,256)是错误的，已修正为实际形状
x = transform(img).unsqueeze(0)
# 打印输入张量形状，方便调试（可选）
print(f"输入张量形状：{x.shape}")  # 输出：torch.Size([1, 1, 28, 28])

# ==========================
# 4. 前向推理（无梯度计算）
# ==========================
# 禁用梯度计算上下文管理器：推理阶段无需计算梯度，提升速度+节省内存
# （原代码未加，此处补充最佳实践）
with torch.no_grad():
    # 将预处理后的图片输入模型，执行前向传播，得到输出
    y = model(x)

# 打印网络输出：形状为(1,10)的张量，每个值是对应数字（0-9）的logits（未归一化的得分）
print("网络输出logits（0-9数字的原始得分）：")
print(y)

# 可选：将logits转为概率（补充实用功能，帮助理解输出）
prob = F.softmax(y, dim=1)  # 对输出维度做Softmax归一化，得到概率
print(f"\n各数字的预测概率：{prob.squeeze().numpy()}")
```

1.5、加宽

> 本小节，了解即可。

增大模型的参数量，一种常见的方法是加宽，在 Inception 结构中，“加宽”是指在同一层中**并行使用多个不同类型的卷积分支**，并在通道方向上将其输出拼接，从而增加特征图的通道数。

这种设计的核心思想是：**在相同层级上同时提取多尺度特征，好处：**

1. **多尺度特征建模**
   不同大小的卷积核对应不同的感受野。
   通过并行的（1×1）、（3×3）、（5×5）（或其等效形式）卷积，网络可以同时捕捉局部细节与较大范围的空间结构。

2. **提高特征表达能力**
   加宽并不会改变网络深度，但显著增加了同一层可学习的特征种类，使模型在不加深结构的情况下获得更强的表示能力。
3. **缓解单一路径的表示瓶颈**
   多分支结构避免了所有特征都必须通过同一卷积路径，有助于降低信息丢失风险。。

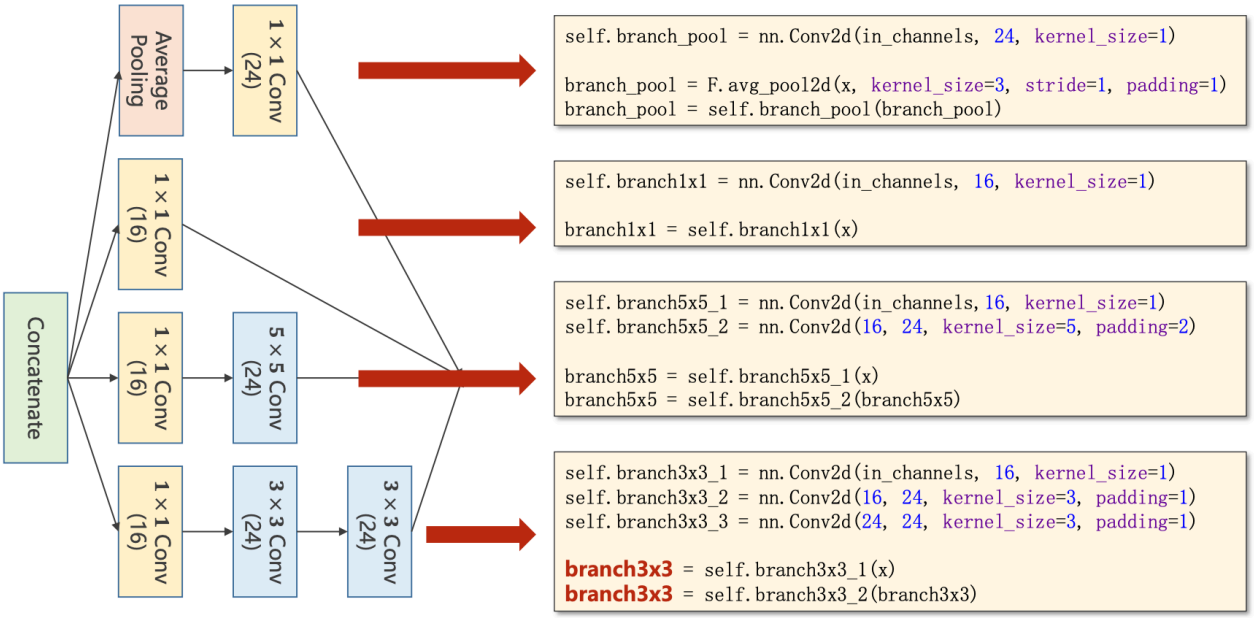


图1-15 inception结构示意图

1.6、加深

关于加深网络层数的重要性，目前的理论研究仍不充分。尽管如此，已有的实验结果和经验性研究表明，加深网络结构在实践中具有显著优势。本节将对加深层的动机和意义作简要说明。

首先，从以 ILSVRC（ImageNet）为代表的大规模图像识别竞赛结果可以清楚地观察到**网络加深的趋势**。近年来，排名靠前的方法几乎均基于深度学习模型，并且网络结构不断加深。总体而言，随着网络层数的增加，模型的识别性能也随之提升。

其次，加深网络层数有助于**提高模型的表示能力**。通过逐层堆叠非线性变换，网络可以构建更加复杂、更加抽象的特征表示，从而更有效地刻画输入数据中的结构信息。

因此，在保持计算效率的同时，加深网络结构能够提升模型的表达能力和性能，这也是现代卷积神经网络普遍采用深层结构的重要原因。

但是观察图 1-16 可以发现，当网络层数从 20 层继续加深到 56 层时，模型的错误率非但没有进一步下降，反而出现了明显上升。这一现象表明，**简单地堆叠网络层并不能保证性能持续提升**，深层网络在实际训练过程中会面临新的困难。

这种性能退化并非由过拟合引起，因为更深的网络在训练集上的误差同样更高；其根本原因在于**深层网络的优化难度显著增加**，例如梯度消失、梯度爆炸以及信息在多层传递过程中逐渐衰减等问题，使得网络难以有效学习到理想的映射关系。

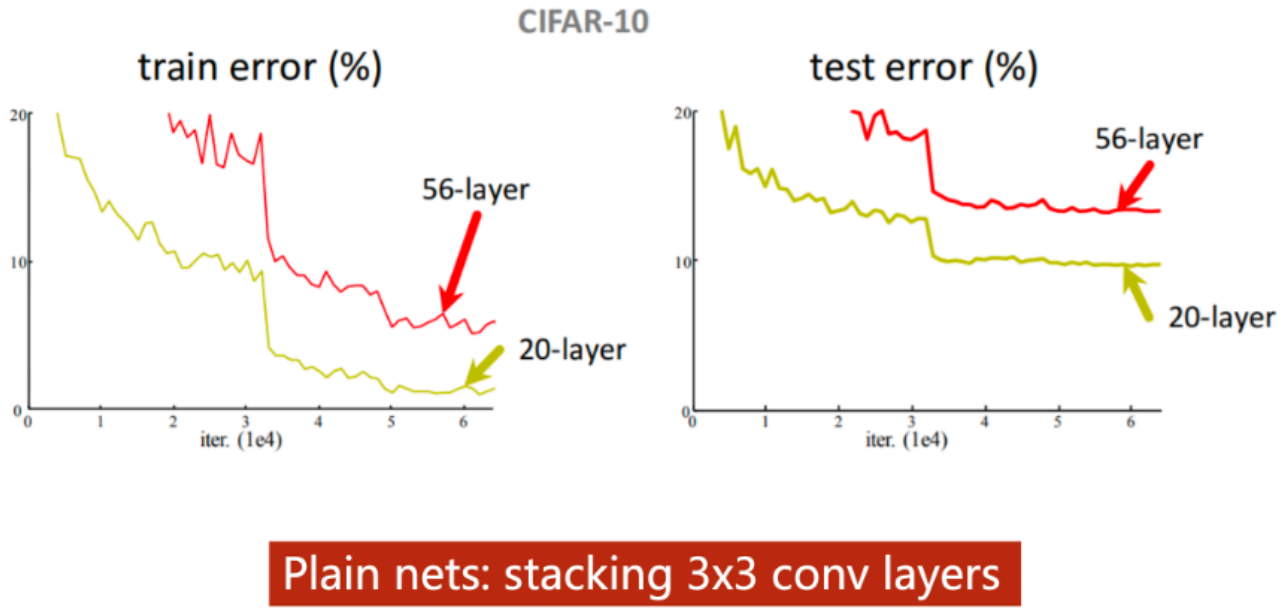


图1-16 随着网络层数的加深错误率的变换

为什么深层网络中会出现梯度消失与梯度爆炸

在深度神经网络的训练过程中，模型参数通常通过**反向传播算法**进行更新。反向传播的本质是**链式法则的连续应用**：梯度需要从输出层逐层传递回输入层。在这一过程中，梯度会不断与各层的权重矩阵及激活函数的导数相乘。
当网络层数较深时，如果这些连乘项的模普遍**小于 1**，梯度在多次传递后会迅速衰减，从而导致**梯度消失**，使得靠近输入端的参数几乎得不到有效更新；反之，如果连乘项的模普遍**大于 1**，梯度会在传播过程中被不断放大，最终造成**梯度爆炸**，使训练过程不稳定甚至无法收敛。

1.7、ImageNet

> 了解

- 数据集

ImageNet 是由李飞飞教授团队从 2007 年开始组织构建的大规模视觉图像数据库(示例如图1-17所示），并于 2009 年正式发布，为计算机视觉研究提供了丰富的带标签图像资源。基于 ImageNet 数据集子集的 ImageNet 大规模视觉识别挑战赛（ILSVRC）从 2010 年起成为评估图像分类和目标识别算法性能的重要基准。在这一竞赛中，深度卷积神经网络成绩显著优于传统方法，并推动了神经网络设计的快速发展。

ImageNet 的主要特点如下：

- 包含上百万张标注图像；
- 覆盖上千个物体类别；
- 每张图像均配有人工标注的类别标签。

在最常用的 ImageNet Large Scale Visual Recognition Challenge（ILSVRC）设置中，数据集包含约 **1000 个类别**，训练集规模约 **120 万张图像**，验证集和测试集各包含约 5 万张图像。

图 1-17 大规模数据集 ImageNet 的数据例

- ImageNet 挑战赛（ILSVRC）

ImageNet 数据集催生了 ImageNet 大规模视觉识别挑战赛（ILSVRC），该竞赛主要评估模型在以下任务上的性能：

- 图像分类（Classification）
- 目标定位（Localization）
- 目标检测（Detection）

其中，图像分类任务最具代表性，通常以 **Top-1** 和 **Top-5 错误率**作为评价指标。ILSVRC 为不同算法提供了公平、可复现的比较环境，对推动计算机视觉算法的发展起到了关键作用。

- **ImageNet 对计算机视觉发展的影响**
    - **推动深度学习方法的崛起**

在 ImageNet 之前，计算机视觉任务主要依赖人工设计的特征，如 SIFT、HOG 等。随着 ImageNet 提供的大规模标注数据，深层卷积神经网络得以充分训练，其性能显著超过传统方法。2012 年，基于深度卷积神经网络的 AlexNet 在 ImageNet 分类任务中取得了突破性成果，这一事件被广泛视为**深度学习在计算机视觉领域的重要转折点。**

- **促进网络结构的快速演进**

ImageNet 为复杂模型的训练提供了充足的数据支持，使研究者能够探索更深、更复杂的网络结构。许多具有代表性的卷积神经网络（如 VGG、GoogLeNet、ResNet）均是在 ImageNet 任务中提出并验证其有效性的。这些模型不仅提升了分类性能，也对后续目标检测、语义分割等任务产生了深远影响。

- **推动迁移学习与预训练范式**

由于 ImageNet 数据规模大、类别丰富，基于 ImageNet 训练得到的模型能够学习到具有较强泛化能力的视觉特征。因此，“**ImageNet 预训练 + 下游任务微调**”逐渐成为计算机视觉领域的常用范式，显著降低了训练成本，并提升了模型在小数据集上的表现。

实践中经常会灵活应用使用 ImageNet 这个巨大的数据集学习到的权重数据，这称为迁移学习，将学习完的权重(的一部分)复制到其他神经网络，进行再学习(fine tuning)。比如，准备一个和 ResNet18 相同结构的网络，把学习完的权重作为初始值，以新数据集为对象，进行再学习。迁移学习在手头数据集较少时非常有效。

1.8、代表性的CNN网络

> 了解即可，不需要深究，好多网络都过时了。
>
> 自 2012 年 AlexNet 在 ILSVRC 上以明显优势获胜以来，多个深度网络架构在 ImageNet 上取得了突破性进展。例如，VGGNet 通过更深的层数和统一的卷积核设计提升了分类精度，GoogLeNet（Inception）提出了更高效的模块化结构，而 ResNet 在 2015 年凭借残差连接机制有效训练了数十层乃至上百层深度网络并取得了优异成绩。这些模型不仅不断刷新 ImageNet 的性能记录，也成为现代卷积神经网络设计的重要里程碑。

在前面的小节中，我们介绍了卷积层、池化层等基本构成要素。本节将对几种**具有代表性的卷积神经网络（CNN）结构**进行简要说明，以帮助理解 CNN 的发展脉络及其设计思想。

1.8.1 LeNet

LeNet 是最早成功应用于实际问题的卷积神经网络之一，主要用于手写数字识别任务。其网络结构由**卷积层、池化层和全连接层交替堆叠**构成，体现了 CNN 的基本设计范式。

LeNet 的特点在于：

- 使用局部连接和权值共享的卷积层；
- 通过池化层逐步降低特征图的空间分辨率；
- 在网络末端使用全连接层进行分类。

尽管结构较为简单，但 LeNet 奠定了现代 CNN 的基本框架。

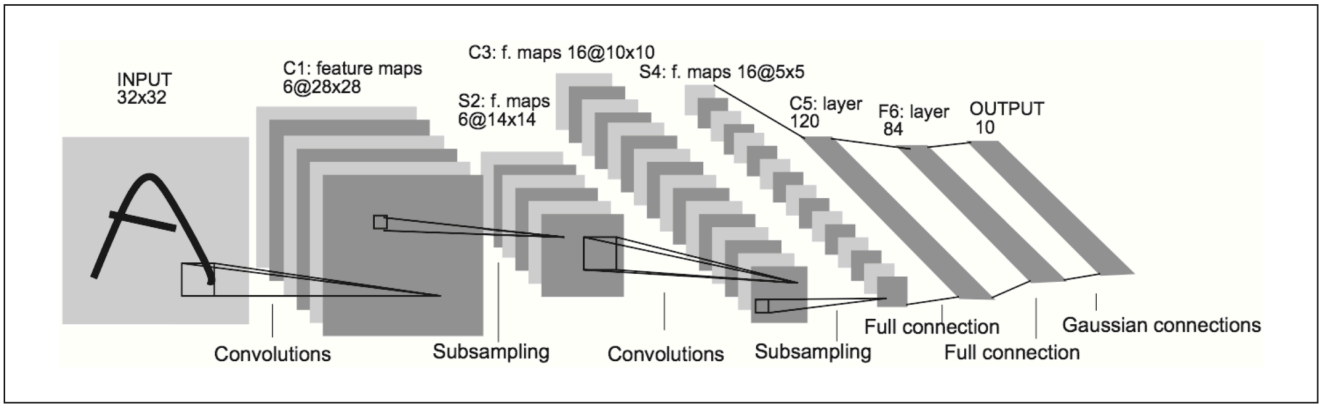


图1-18 LeNet网络结构

1.8.2 AlexNet

AlexNet 是在大规模图像分类任务中取得突破性成果的网络结构。与 LeNet 相比，AlexNet 具有**更深的网络结构和更大的模型容量**，并在多个方面进行了改进。

AlexNet 的主要特点包括：

- 使用 ReLU 作为激活函数，加快了训练收敛速度；
- 采用更大的卷积核和更深的网络层数；

- 在训练过程中引入 Dropout 以缓解过拟合问题。

AlexNet 的成功推动了深度学习在计算机视觉领域的广泛应用。

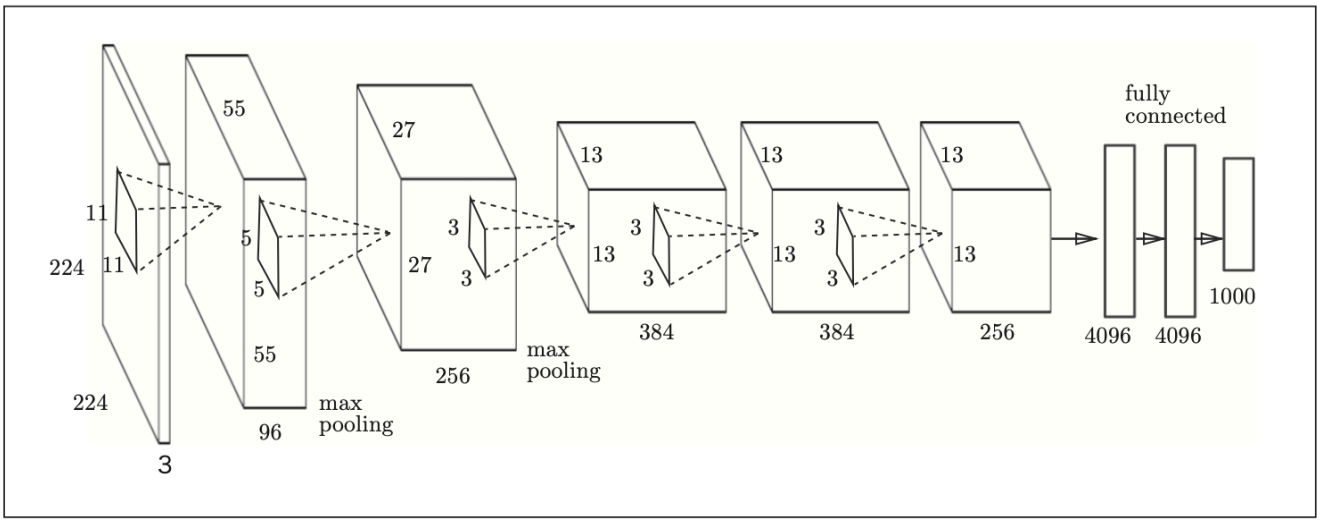


图1-19 AlexNet网络结构

1.8.3 VGG

VGG 网络通过系统性地研究网络深度对性能的影响，提出了**使用多个小卷积核（如 (**3×3**)）替代大卷积核**的设计思想。

VGG 的主要特点是：

- 网络结构规则、层次清晰；
- 通过堆叠多个 (3×3) 卷积层来扩大感受野；
- 参数数量较大，但结构易于理解和实现。

由于其简洁性，VGG 常被用于特征提取和迁移学习任务。

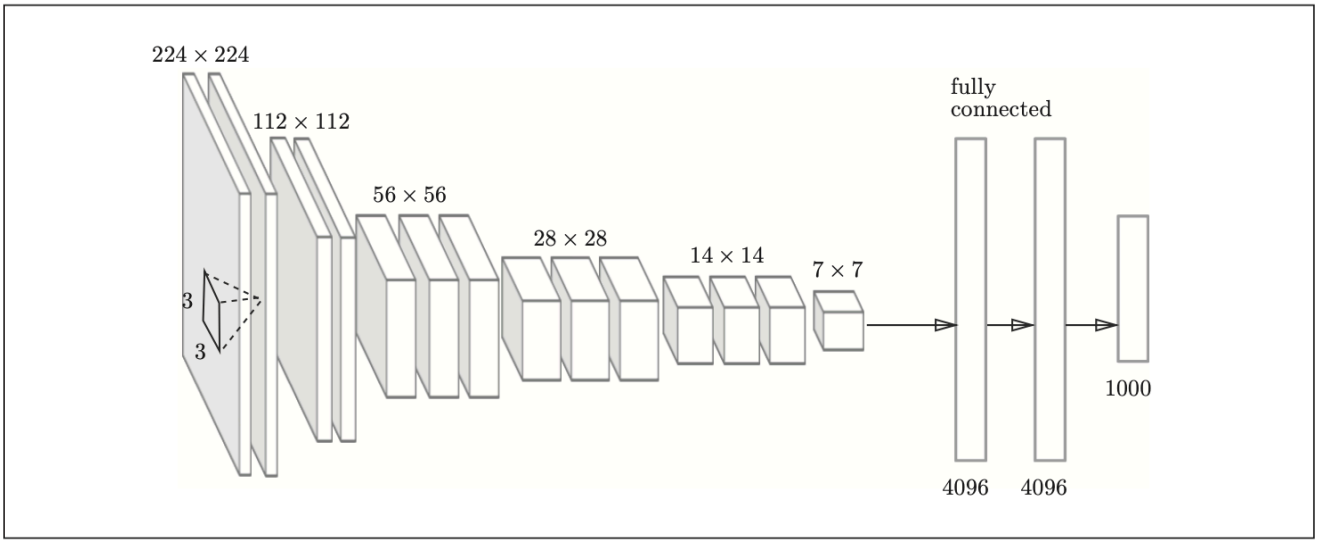


图1-20 VGG网络结构

1.8.4 GoogLeNet（Inception）

GoogLeNet 通过引入 Inception 模块，在保持网络深度的同时**显著减少了参数数量。**

Inception 模块的核心思想是：

- 在同一层中并行使用不同尺寸的卷积核；
- 通过 (1×1) 卷积进行降维，降低计算成本；
- 将多尺度特征在通道方向上进行拼接。

该结构提高了网络对不同尺度特征的表达能力。

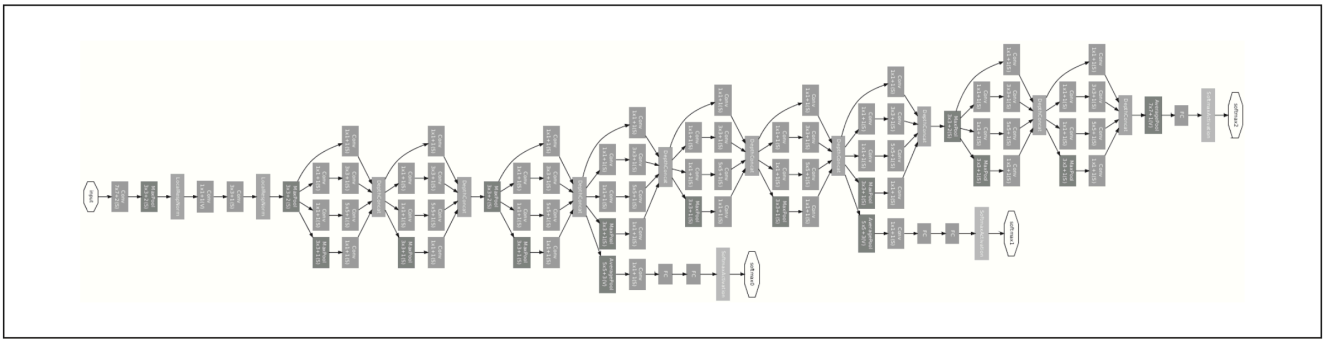


图1-21 GoogleNet网络结构

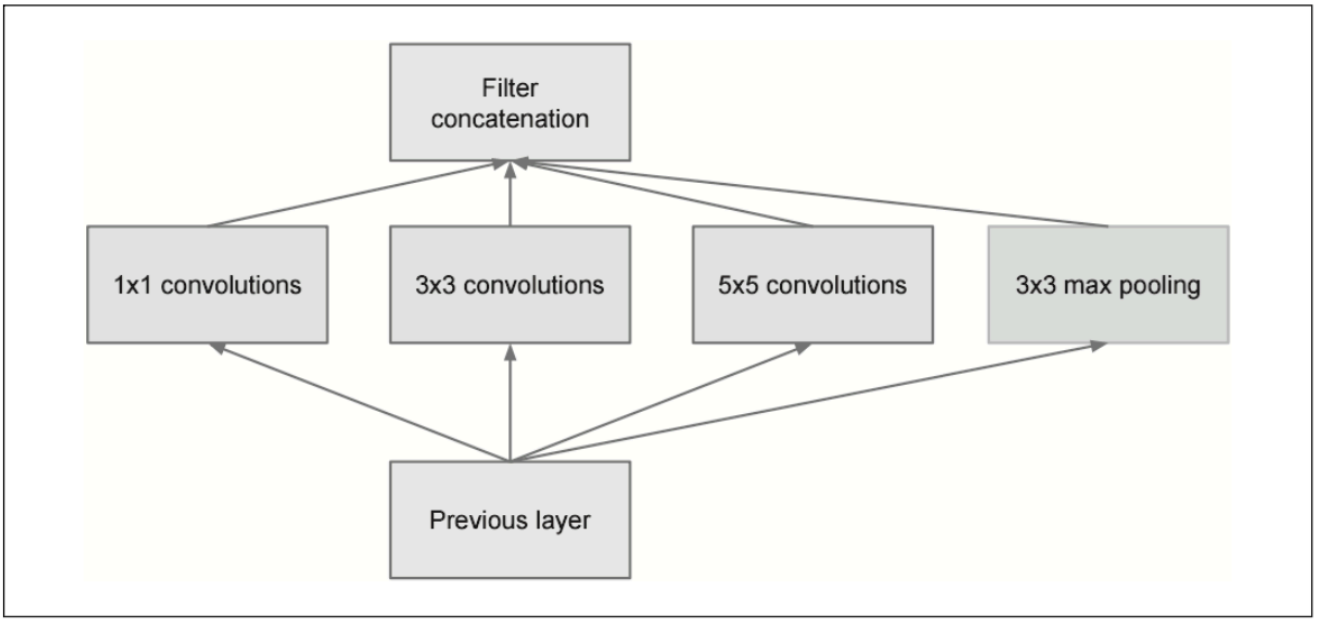


图1-22 GoogleNet中Inception结构

1.8.5 ResNet

随着网络不断加深，训练过程会面临梯度消失等问题。ResNet 通过引入**残差连接（skip connection）**，有效缓解了这一困难。

ResNet 的核心思想是让网络学习残差映射，即：

$$\mathcal{H}(x) = F(x) + x$$

这种结构使得信息和梯度可以在网络中直接传播，从而支持构建非常深的网络结构。

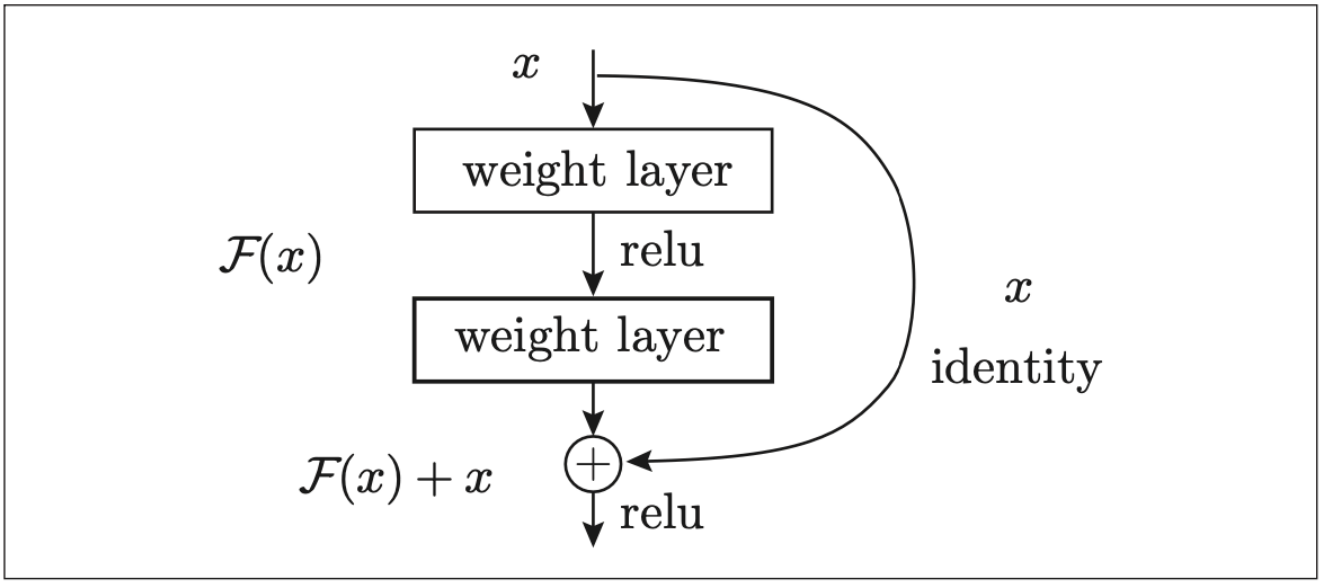


图1-23 ResNet的构成要素:这里的“weight layer”是指卷积层

因为快捷结构只是原封不动地传递输入数据，所以反向传播时会将 来自上游的梯度原封不动地传向下游。这里的重点是不对来自上游 的梯度进行任何处理，将其原封不动地传向下游。因此，基于快捷 结构，不用担心梯度会变小(或变大)，能够向前一层传递“有意义 的梯度”。通过这个快捷结构，之前因为加深层而导致的梯度变小的 梯度消失问题就有望得到缓解。

从 LeNet 到 ResNet，卷积神经网络的结构不断演进，但其核心思想始终围绕以下几点展开：

利用卷积操作提取局部特征；

通过层次化结构逐步抽象高层语义；

在保证表达能力的同时控制模型复杂度。

1.9、残差块

为了解决深层网络训练困难的问题，He 等人提出了**残差学习（Residual Learning）的思想，并在此基础上构建了残差网络（ResNet）**。其核心思想并非直接学习期望的映射 (H(x))，而是引导网络学习残差函数 (F(x)=H(x)-x)，并通过**捷径连接（Shortcut Connection）将输入 (x) 直接与输出相加，从而得到**
y = F(x) + x
这种结构被称为残差块（Residual Block），如图1-24所示。残差块的引入使得信息和梯度能够在网络中更加顺畅地传播，即使在网络不断加深的情况下，也能有效缓解训练过程中的退化问题。当残差分支学习到的映射趋近于零时，残差块至少可以实现恒等映射，从而保证深层网络的性能不劣于浅层网络。

因此，残差块为构建超深神经网络提供了一种有效且稳定的结构设计，使得网络层数可以扩展到数十层甚至上百层，同时仍能保持良好的收敛性和性能表现。这一思想也成为现代深度卷积神经网络设计中的重要基础。

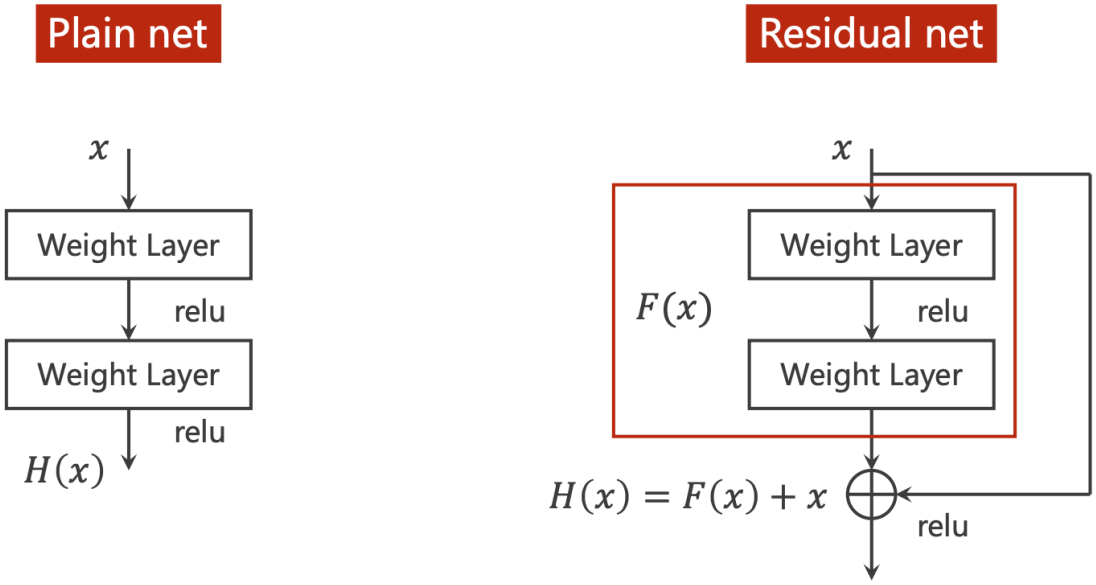
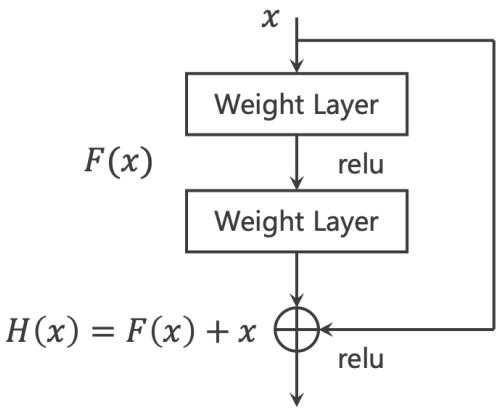


图1-24 残差网络结构示意图




```python
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super(ResidualBlock, self).__init__()
        self.channels = channels
        self.conv1 = nn.Conv2d(channels, channels,
                               kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(channels, channels,
                               kernel_size=3, padding=1)

    def forward(self, x):
        y = F.relu(self.conv1(x))
        y = self.conv2(y)
        return F.relu(x + y)
```

图1-25 残差网络结构示意图和对应torch代码

> 对应文件：residual_block.py

```python
# 导入必要的PyTorch模块
import torch
import torch.nn as nn
import torch.nn.functional as F

# ========================= 残差块（Residual Block）完整实现 =========================
class ResidualBlock(nn.Module):
    """
    基础残差块（恒等映射版）：输入输出通道数相同，适用于特征图尺寸不变的场景
    结构：Conv2d(3x3) → ReLU → Conv2d(3x3) → 残差连接（x+y） → ReLU
    """
    def __init__(self, channels):
        super(ResidualBlock, self).__init__()
        self.channels = channels  # 输入/输出通道数（残差块保持通道数不变）

        # 第一个卷积层：3x3卷积，padding=1保证特征图尺寸不变
```

```
        self.conv1 = nn.Conv2d(
            in_channels=channels,
            out_channels=channels,
            kernel_size=3,
            padding=1,  # padding=1 抵消kernel_size=3的尺寸缩减，保持H/W不变
            bias=False  # 残差块中通常配合BN使用（本示例简化未加BN，可保留bias）
        )

        # 第二个卷积层：与conv1参数一致，保持通道数和尺寸不变
        self.conv2 = nn.Conv2d(
            in_channels=channels,
            out_channels=channels,
            kernel_size=3,
            padding=1,
            bias=False
        )

    def forward(self, x):
        """
        前向传播：残差连接核心逻辑
        :param x: 输入张量，形状为 [batch_size, channels, height, width]
        :return: 输出张量，形状与输入完全一致
        """
        # 主干路径：卷积1 → ReLU激活
        y = F.relu(self.conv1(x))
        # 主干路径：卷积2（无激活，留到残差连接后）
        y = self.conv2(y)
        # 残差连接：输入x + 主干输出y → 再激活（符合残差网络的经典设计）
        y = F.relu(x + y)

        return y
```

残差结构为什么可以缓解梯度消失和梯度爆炸

残差块通过引入**恒等捷径连接（Identity Shortcut）**，使网络的前向传播和反向传播路径中都额外增加了一条**不经过非线性变换的直接通路**。在反向传播时，梯度不仅通过残差分支进行传递，还可以**直接沿捷径连接回传**，避免了梯度必须连续经过多层权重和非线性函数的情况。
从数学角度看，残差块的输出形式为： $y = F(x) + x$
其梯度可以表示为: $\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + I$
其中 (I) 表示恒等映射对应的梯度项。该项的存在保证了梯度在反向传播过程中**至少包含一条稳定的恒等传递路径**，从而显著降低梯度消失和梯度爆炸发生的概率。

1.10、简单的残差网络

如图1-26所示，我们将残差块引入到网络中，构建一个简单的残差网络。

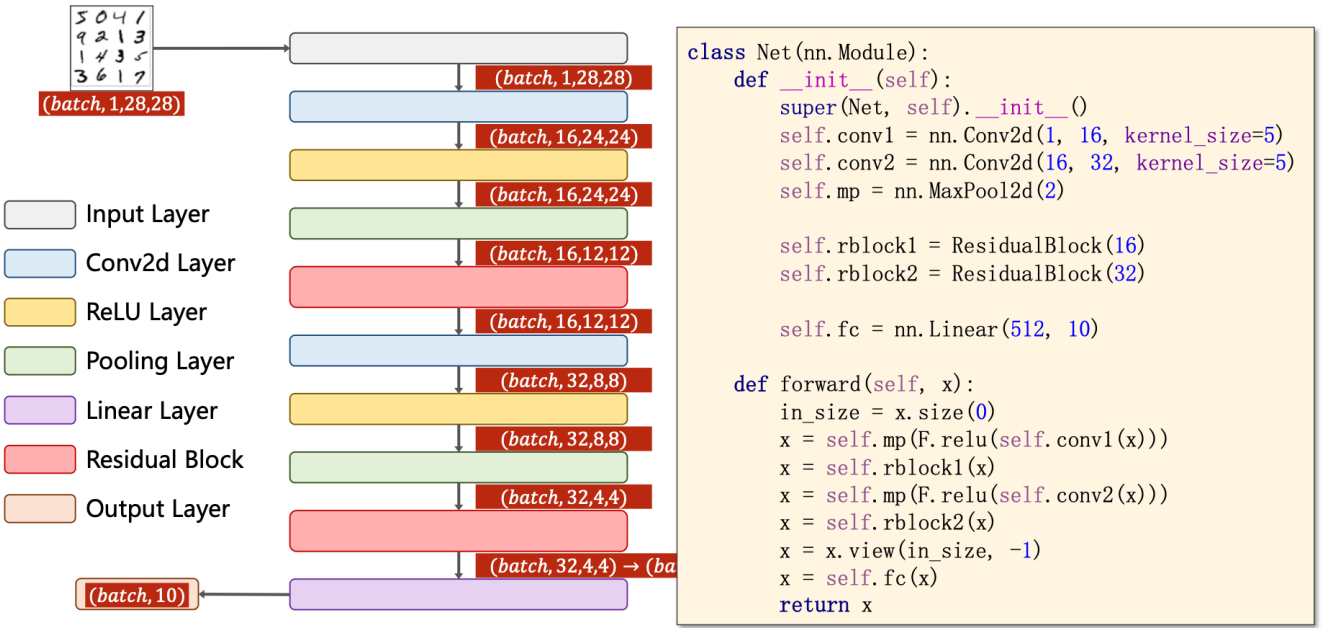


图1-26 引入残差块的简单网络示意图

> 对应文件：residual_net.py

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=5)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=5)
        self.mp = nn.MaxPool2d(2)
        self.rblock1 = ResidualBlock(16)
        self.rblock2 = ResidualBlock(32)
        self.fc = nn.Linear(512, 10)
    def forward(self, x):
        in_size = x.size(0)
        x = self.mp(F.relu(self.conv1(x)))
        x = self.rblock1(x)
        x = self.mp(F.relu(self.conv2(x)))
        x = self.rblock2(x)
        x = x.view(in_size, -1)
        x = self.fc(x)
        return x
```

1.11、ResNet网络

> 知道使用残差块来堆叠起来的就行，不需要知道网络里面每一层的细节，我们在使用的时候，也是直接调用torchvision库。

残差网络（Residual Network，ResNet）（见图1-27）是一类基于残差学习思想构建的深层卷积神经网络结构，由He 等人在 ILSVRC 竞赛中提出。该网络通过在整体架构中系统性地引入跨层连接，使网络能够在保持较高深度的同时仍具备良好的可训练性，从而突破了传统卷积神经网络在层数不断增加时性能退化的瓶颈。

ResNet 的提出显著推动了深度神经网络向更深层次发展，其网络深度可扩展至数十层甚至上百层，并在大规模图像分类等任务中取得了优异效果。由于其结构简洁、训练稳定且具有良好的泛化能力，ResNet 已成为后续众多深度网络模型（如 DenseNet、ResNeXt 等）的重要设计基础，在计算机视觉领域得到了广泛应用。

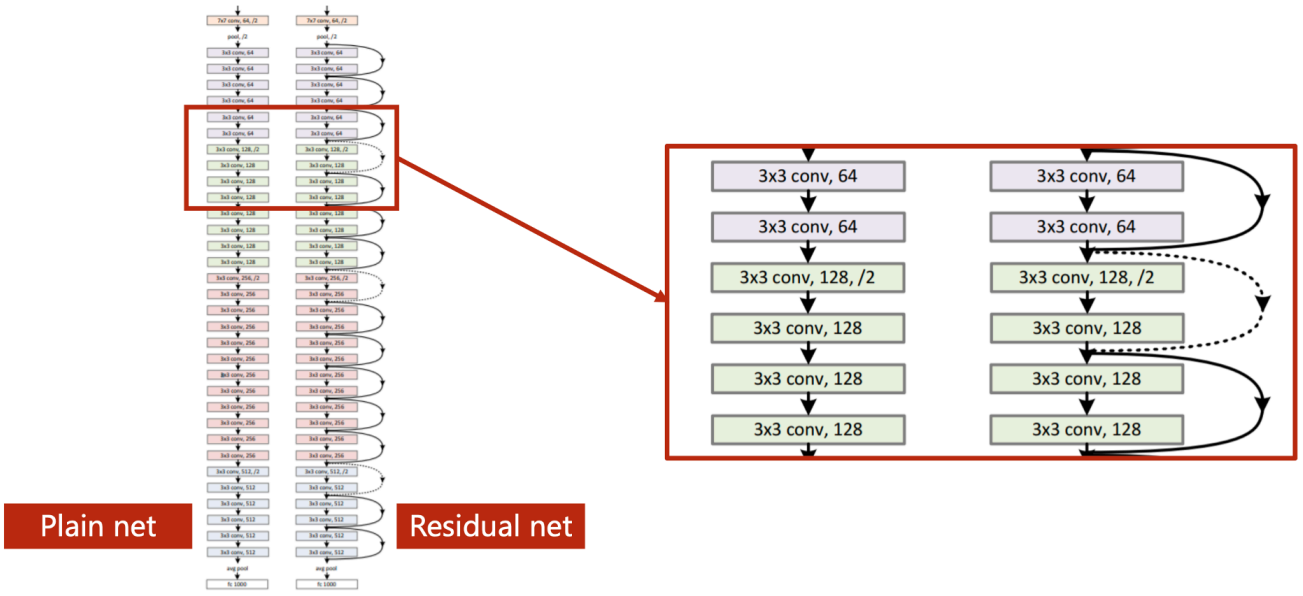


图1-27 残差网络示意图

> 对应文件：resnet.py

```
import torch
import torch.nn as nn
from torchvision import models

# ========================= 1. 基础配置 =========================
# 设置设备：优先使用GPU，无GPU则用CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"使用设备: {device}")

# 分类类别数（示例为10分类）
num_classes = 10

# ========================= 2. 加载并修改 ResNet34 =========================
# 加载预训练的 ResNet34（注意：torchvision 新版本中 pretrained=True 已弃用，改用 weights 参数）
# 兼容新旧版本的写法：
try:
    # 新版本（torchvision >= 0.13）
    from torchvision.models import ResNet34_Weights
    model = models.resnet34(weights=ResNet34_Weights.IMAGENET1K_V1)
except ImportError:
    # 旧版本
    model = models.resnet34(pretrained=True)

# 修改最后一层全连接层（fc）：适配10分类任务
# model.fc.in_features 是原fc层的输入特征数（ResNet34固定为512）
model.fc = nn.Linear(model.fc.in_features, num_classes)

# 将模型移到指定设备（GPU/CPU）
```

```
model = model.to(device)

# 设置模型为评估模式（推理时必须！禁用Dropout/BatchNorm的训练行为）
model.eval()

# ========================= 3. 构造测试输入（模拟RGB图片） =========================
# ResNet34 要求的输入规范：
# - 尺寸：至少224x224（ImageNet预训练模型的标准输入）
# - 通道：3通道（RGB）
# - 形状：[batch_size, channels, height, width]
batch_size = 2  # 模拟批量大小为2的输入
test_input = torch.randn(batch_size, 3, 224, 224).to(device)  # 随机生成测试张量
print(f"测试输入形状: {test_input.shape}")  # 输出: torch.Size([2, 3, 224, 224])

# ========================= 4. 前向推理（核心测试） =========================
# 禁用梯度计算：推理阶段无需计算梯度，提升速度+节省内存
with torch.no_grad():
    # 模型前向传播
    output = model(test_input)

# ========================= 5. 解析输出结果 =========================
print("\n===== 推理结果解析 =====")
# 输出形状：[batch_size, num_classes] → 此处为 [2, 10]
print(f"模型输出形状: {output.shape}")

# 输出原始logits（未归一化的得分）
print("\n原始输出（logits）:")
print(output)

# 将logits转为概率（Softmax归一化）
prob = nn.functional.softmax(output, dim=1)
print("\n归一化后的概率:")
print(prob)

# 预测类别（取概率最大的索引）
pred_labels = torch.argmax(prob, dim=1)
print("\n预测类别索引（0-9）:")
print(pred_labels)

# 验证输出合理性
assert output.shape == (batch_size, num_classes), "模型输出形状错误！"
print("\n✅ 模型测试通过：前向推理正常，输出形状符合预期！")
```

1.12、模型学到了什么?

> 了解即可，不需要深究。

卷积神经网络在训练过程中学习到的，并非具体样本本身，而是对输入数据的**特征表示。**这些特征由网络参数所定义，并以特征图的形式在各层中逐级表达。

在网络的浅层，卷积层主要学习对**局部、低级结构**敏感的特征，如边缘、方向性纹理等。这些特征与经典图像处理中的滤波器具有相似功能。

随着网络层数的加深，感受野逐渐扩大，中间层开始学习由多个低级特征组合而成的**形状和局部部件特征**。在高层，特征表示进一步抽象，与具体的**语义概念和类别判别信息**高度相关。此时，特征更关注“是什么”，而非“在哪里”。

通过卷积、非线性变换和下采样的反复堆叠，CNN 形成了由低级到高级的**层次化特征表示**，这是其在视觉任务中取得良好性能的根本原因。

需要注意的是，模型学习到的是对训练目标最有判别力的统计模式，而非对现实世界的真实理解。

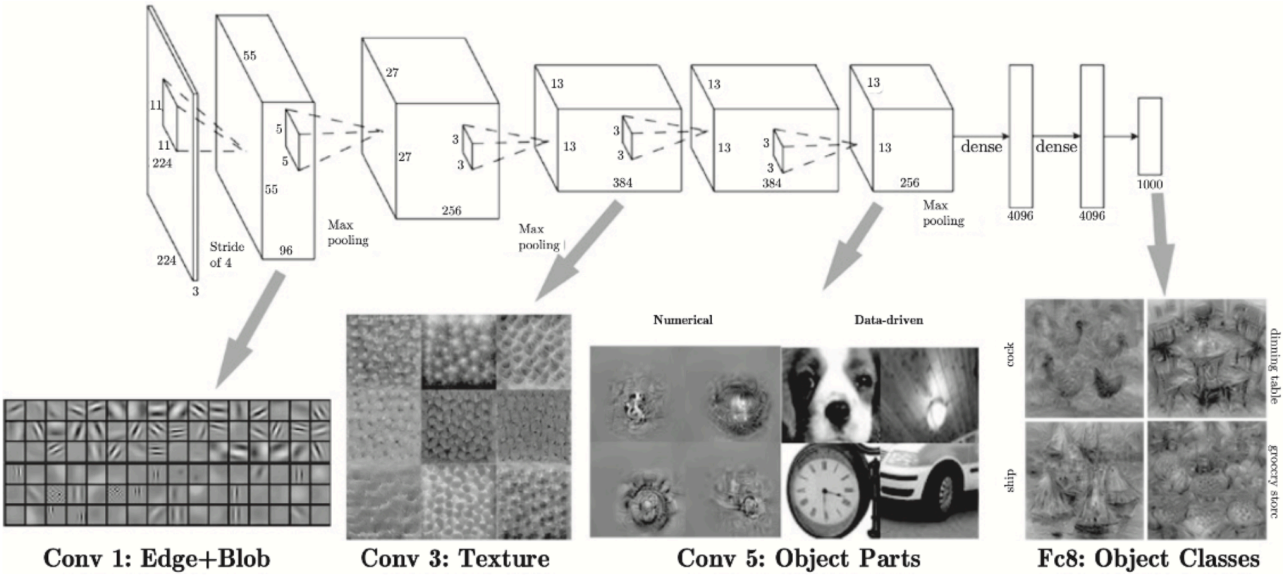
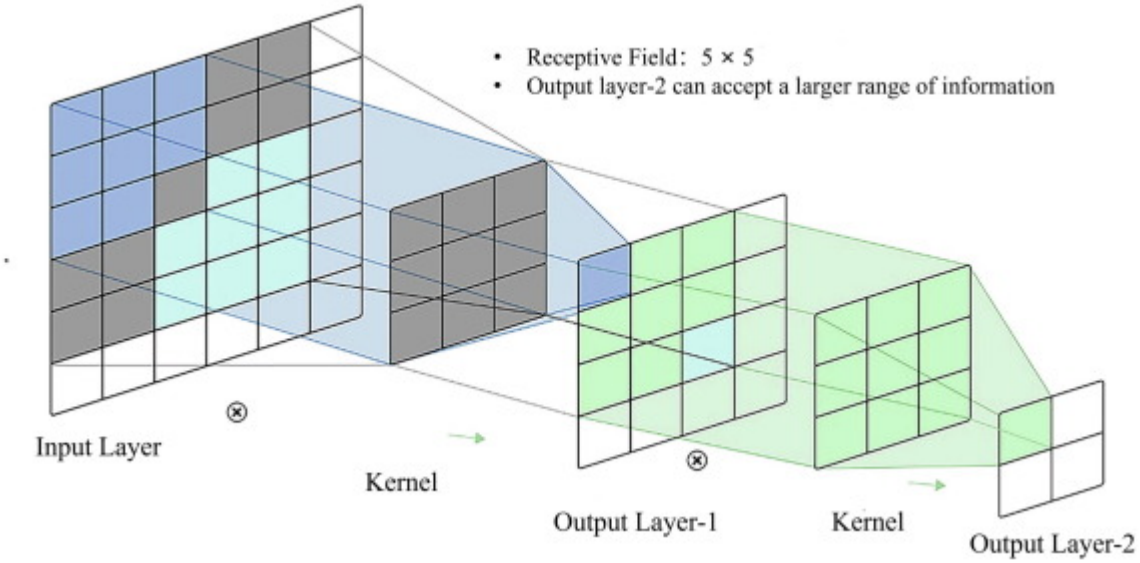


图7-28 CNN的卷积层中提取的信息。第1层的神经元对边缘或斑块有响应，第3层对纹理有响应，第5层对物体部件有响应，最后的全连接层对物体的类别(狗或车)有响应

1.13 QA

- 我们的目标是取得数据集上更好的性能，因此我们会使用现成的网络。至于网络为什么这么设计出来的，我们不需要深究。
- 什么是感受野？以及为什么随着层数的增加会变大？
    - 感受野是指网络中某一层（或某一神经元）在原始输入空间中所对应的有效感知区域，用于衡量该神经元对输入信息的空间依赖范围。



- 代码从上到下的顺序就是模型训练的顺序吗？

    - 代码顺序是**模型构建顺序**，而不是训练过程中数据流动的真实执行顺序。
- 代码需要掌握到什么程度？重点应放在**理解网络结构与数据流动机制**，而非死记语法或API细节。
    - 我们的目标是取得更好的精度，因此需要掌握，三个层次：
    1. **理解层**：能够看懂网络结构定义、损失函数、优化器配置；
    2. **应用层**：能够基于已有代码修改网络结构、超参数、数据加载方式；
    3. **分析层**：能够定位训练问题（如不收敛、过拟合、维度错误），并进行调试。
- 有点分不清池化，卷积，激活函数，归一化这些的顺序和步骤，谁在先谁在后？
    - **卷积**：提取局部特征；
    - **归一化（BN）**：稳定特征分布，加速训练；
    - **激活函数**：引入非线性；
    - **池化**：降低空间分辨率，增强平移不变性。
    - 因此，两个线性变换之间必须有非线性激活函数。其他的没那么固定，尝试出来这个效果好就这样了。
- 看鱼书上说，输入是个四维，输出也是四维的，处理过程中需要对RGB切片处理，变成二维的？$ (N, C, H, W) = (\text{批量大小}, \text{通道数}, \text{高度}, \text{宽度})$例如：((256, 3, 224, 224))。在卷积运算中：因此，虽然计算过程中涉及对各通道的“切片处理”，但这属于**卷积操作的内部机制**，对使用者是透明的，不需要显式将输入拆分为二维矩阵。
    - 是的，但这是**网络内部自动完成的处理方式**，无需人工拆分。对于图像输入，一般表示为四维张量：
    - 网络会对每个通道（如 R、G、B）分别进行卷积运算；
    - 然后在通道维度上进行加权求和，生成新的特征图；
    - 输出仍保持四维结构：((N, C', H', W'))。

二、项目

> 代码地址：链接: https://pan.baidu.com/s/1BZIoYkatVT1NrI8MFBkmag?pwd=6vcr 提取码: 6vcr

2.1、项目介绍

该项目是计算机视觉与模式识别大会(CVPR)细粒度视觉分类(FGVC7)挑战赛的一部分，同时也在kaggle进行数据集下载和提交：kaggle 地址

2.1.1、问题陈述

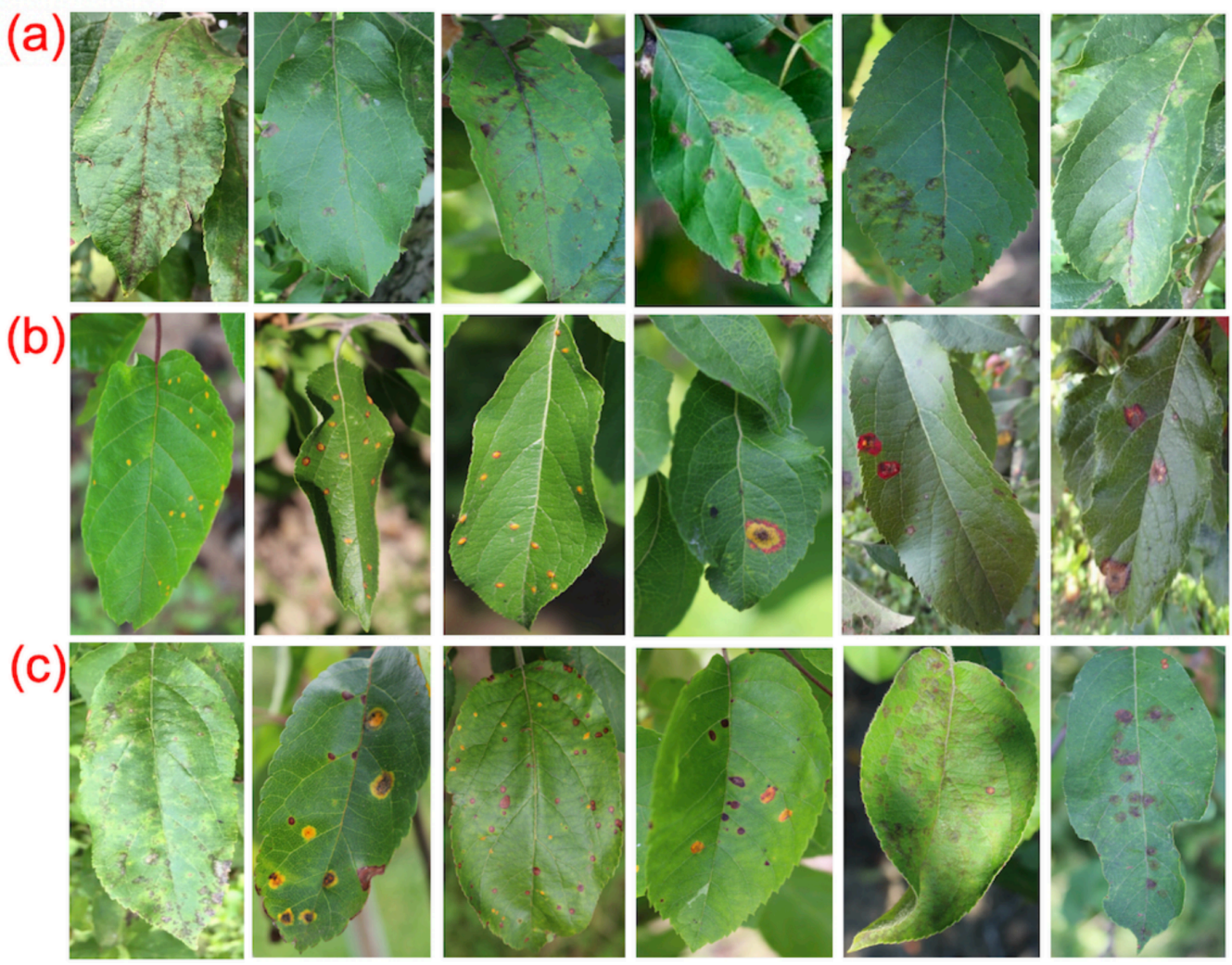
对影响农作物的多种病害的误诊会导致化学品滥用，进而引发抗药菌株的出现、投入成本的增加以及更多疫情的爆发，造成重大的经济损失和环境影响。目前基于人工巡查的病害诊断方法耗时耗力，虽然基于计算机视觉的模型有望提高效率，但由于受感染组织的年龄、遗传变异以及树木内部光照条件等因素，症状差异很大，从而降低了检测的准确性。

2.1.2、具体目标

“植物病理学挑战” 的目标是使用训练数据集的图像来训练一个图片分类模型，以实现：将测试数据集中的给定图像准确分类到不同的病害类别或健康叶片类别中，共4分类，a）雪松苹果锈病，b）苹果黑星病，c）单张叶片上的多种病害，及d）正常的叶片。

2.1.3、数据集来源

本次竞赛是计算机视觉与模式识别大会(CVPR 2020)细粒度视觉分类(FGVC7)挑战赛的一部分，数据集训练集1274、验证集273、测试集274，



2.1.4、评价指标

top1-acc.

Top-1 Accuracy 表示：**模型预测的“最可能类别”与真实类别完全一致的样本比例。**

- 若模型对某样本预测的概率最高的类别就是正确类别 → 记为 1；
- 否则 → 记为 0；

```
# 模型验证组件（核心功能：模型性能评估）
def test(model, test_loader, criterion, device):
    """模型测试流程（关键区别：无梯度计算）"""
    model.eval()  # 切换评估模式（关闭Dropout等）
    correct = 0
    total = 0
    running_loss = 0.0

    with torch.no_grad():
```

```
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            # 前向传播
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            # 统计指标
            running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    test_loss = running_loss / len(test_loader)
    test_acc = correct / total
    print(f'Test Loss: {test_loss:.3f}, Accuracy: {test_acc*100:.2f}%')
    return test_loss, test_acc
```

2.2、数据集

2.2.1、数据集划分

每个类别的图片在对应的文件夹下，我们将数据划分为训练、验证和测试集，

```
dataset/
└── images/
    ├── healthy/
    |   ├── Train_2.jpg
    |   └── 002.jpg
    ├── multiple_diseases/
    |   ├── 001.jpg
    ├── rust/
    |   ├── 003.png
    ├── scab/
    |   ├── 003.png
```

```
import os
import random

# ----------------------------
# 参数配置区
# ----------------------------

images_folder = "./dataset/images"

train_txt = "./dataset/label_file/train.txt"
val_txt   = "./dataset/label_file/val.txt"
test_txt  = "./dataset/label_file/test.txt"

train_ratio = 0.7
```

```
val_ratio   = 0.15
test_ratio  = 0.15

seed = 42   # 固定随机种子，保证可复现

# ---------------------------
# 核心逻辑
# ---------------------------

def generate_txt_labels():
    """
    生成 train / val / test 三个 txt 文件
    每行格式：
    relative_path<TAB>label
    """

    assert abs(train_ratio + val_ratio + test_ratio - 1.0) < 1e-6, \
        "train / val / test 比例之和必须为 1"

    # 1. 获取并排序类别文件夹
    class_names = sorted([
        d for d in os.listdir(images_folder)
        if os.path.isdir(os.path.join(images_folder, d))
    ])

    # 2. 自动生成 label（按 sorted 顺序）
    label_map = {name: idx for idx, name in enumerate(class_names)}

    print("自动生成的类别映射：")
    for k, v in label_map.items():
        print(f"{k} -> {v}")

    # 3. 收集所有样本
    samples = []
    for class_name in class_names:
        class_dir = os.path.join(images_folder, class_name)
        label = label_map[class_name]

        for img_name in os.listdir(class_dir):
            if img_name.lower().endswith((".jpg", ".png", ".jpeg")):
                rel_path = os.path.join(class_name, img_name)
                samples.append((rel_path, label))

    # 4. 固定随机种子并打乱
    random.seed(seed)
    random.shuffle(samples)

    # 5. 按比例切分
    total = len(samples)
    train_end = int(total * train_ratio)
    val_end   = train_end + int(total * val_ratio)

    train_data = samples[:train_end]
```

```
    val_data   = samples[train_end:val_end]
    test_data  = samples[val_end:]

    # 6. 写入 txt 文件
    os.makedirs(os.path.dirname(train_txt), exist_ok=True)

    def write_txt(path, data):
        with open(path, "w") as f:
            for rel_path, label in data:
                f.write(f"{rel_path}\t{label}\n")

    write_txt(train_txt, train_data)
    write_txt(val_txt, val_data)
    write_txt(test_txt, test_data)

    print("\n数据集划分完成：")
    print(f"Total: {total}")
    print(f"Train: {len(train_data)}")
    print(f"Val  : {len(val_data)}")
    print(f"Test : {len(test_data)}")
    print(f"TXT 文件路径: {os.path.dirname(train_txt)}")

if __name__ == "__main__":
    generate_txt_labels()
```

2.2.2、数据集预处理及增强

```
from torchvision import transforms

# 构建增强后的训练集数据变换
train_transform = transforms.Compose([
    # 1. 缩放控制（指定尺寸，替代原随机裁剪的尺寸固定需求）
    transforms.Resize((256, 256)),  # 先统一缩放到256x256，为后续裁剪留空间
    # 2. 随机裁剪缩放（裁剪到目标尺寸224x224）
    transforms.RandomResizedCrop(224, scale=Å),  # scale控制裁剪区域占比（0.8-1.0）
    # 3. 随机翻转（包含水平+垂直，即上下左右翻转）
    transforms.RandomHorizontalFlip(p=0.5),  # 随机水平翻转（左右），概率0.5
    transforms.RandomVerticalFlip(p=0.5),    # 随机垂直翻转（上下），概率0.5
    # 4. 随机颜色增强（亮度=光照，对比度）
    transforms.ColorJitter(
        brightness=(0.5, 1.5),  # 随机光照增强：亮度范围0.5-1.5（<1变暗，>1变亮）
        contrast=(0.5, 1.5)     # 随机对比度增强：对比度范围0.5-1.5
    ),
    # 5. 转为张量+标准化（保留原有功能）
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Step 0: Original Image
- Size: 2048x1365

Step 1: Resize
- Params: size=(256, 256)
- Intensity: Fixed scaling: Resize any input image to 256x256 (no randomness)

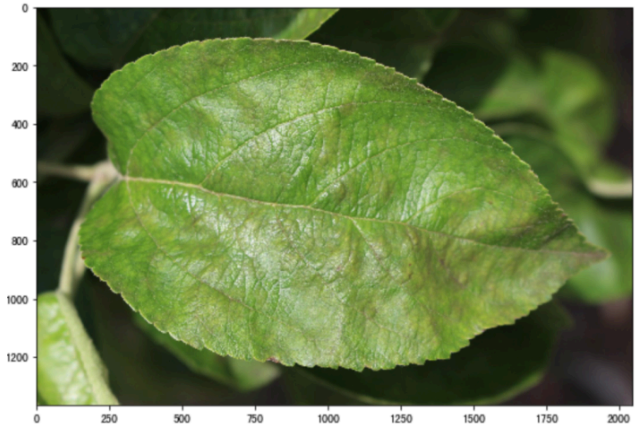
Step 2: RandomResizedCrop
- Params: size=224, scale=0.93 (random in 0.8-1.0)
- Intensity: Crop area ratio: 93% (0.8=80%, 1.0=100%), then resize to 224x224

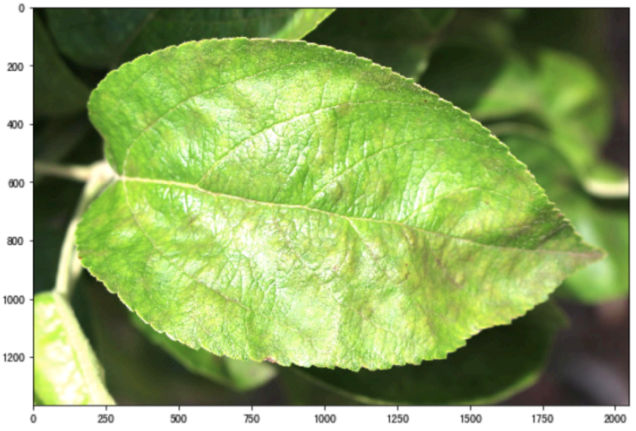
Step 3: RandomHorizontalFlip
- Params: p=0.5, current random prob=0.03
- Intensity: Execution: □ Flipped Horizontally (flip if prob < 0.5)

Step 4: RandomVerticalFlip
- Params: p=0.5, current random prob=0.28
- Intensity: Execution: □ Flipped Vertically (flip if prob < 0.5)

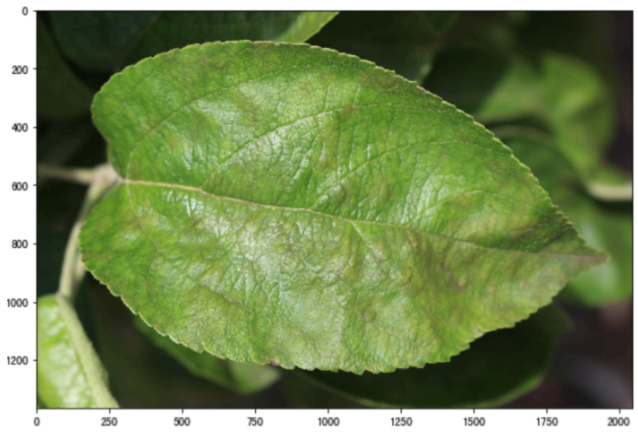
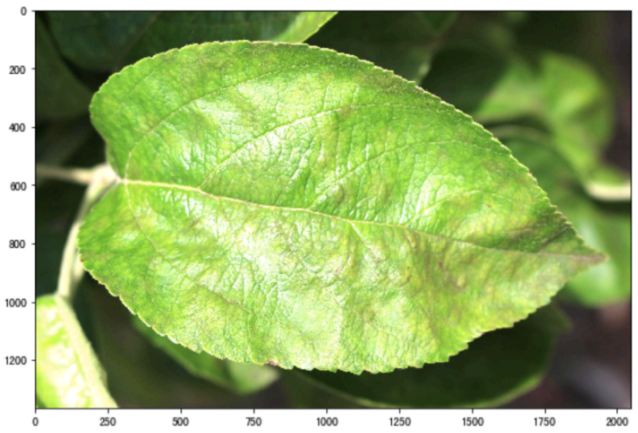
Step 5: ColorJitter
- Params: brightness=0.72 (random 0.5-1.5), contrast=1.24 (random 0.5-1.5)
- Intensity: Brightness decreased by 28% (lower = darker), Contrast increased by 24% (higher = more vivid)

- 随机光照增强

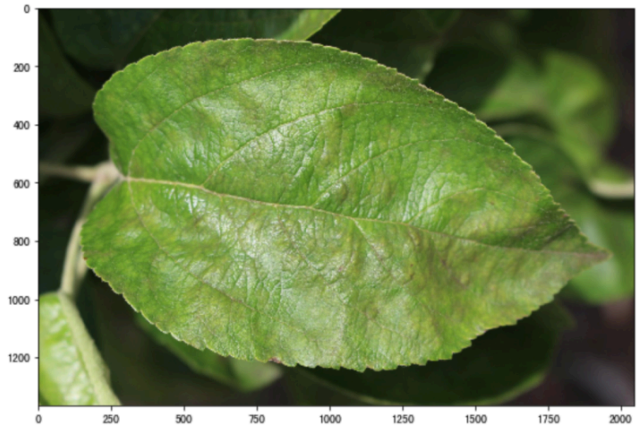
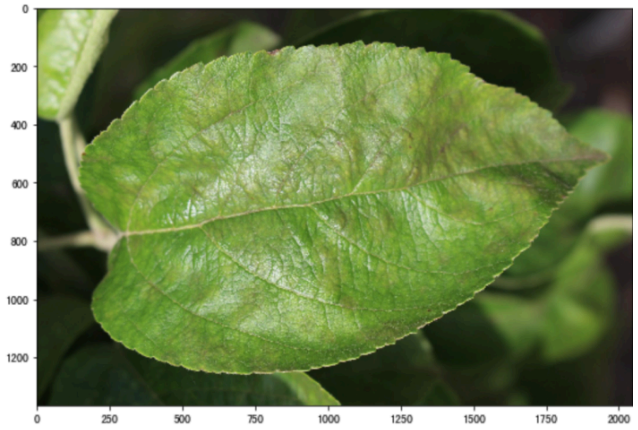

0
200
400
600
800
1000
1200
0
250
500
750
1000
1250
1500
1750
2000


0
200
400
600
800
1000
1200
0
250
500
750
1000
1250
1500
1750
2000

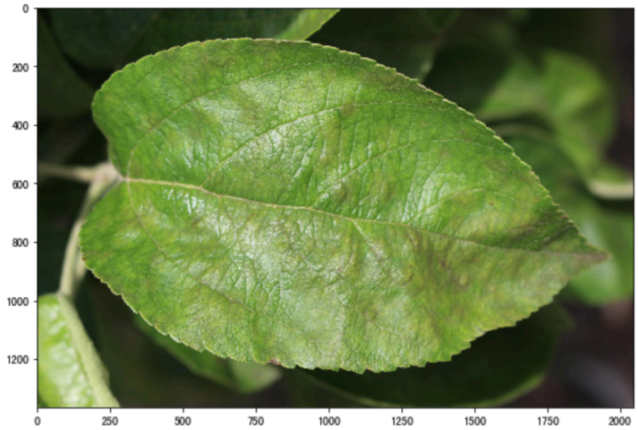
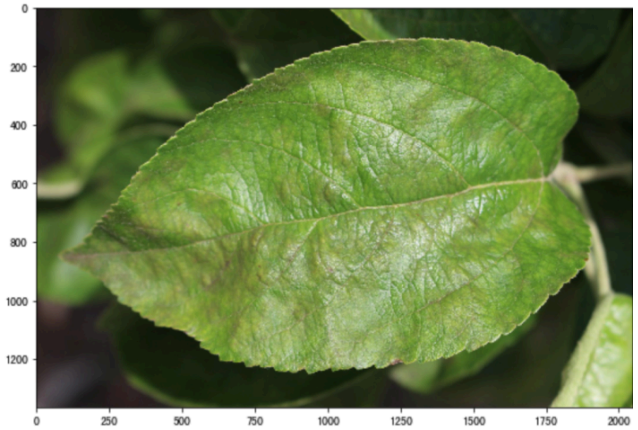
- 随机对比度增强

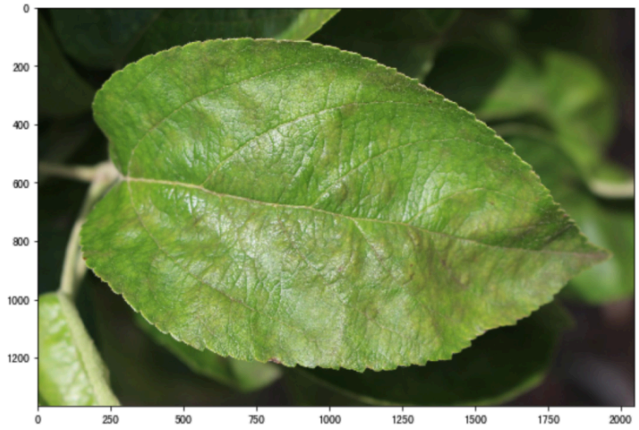
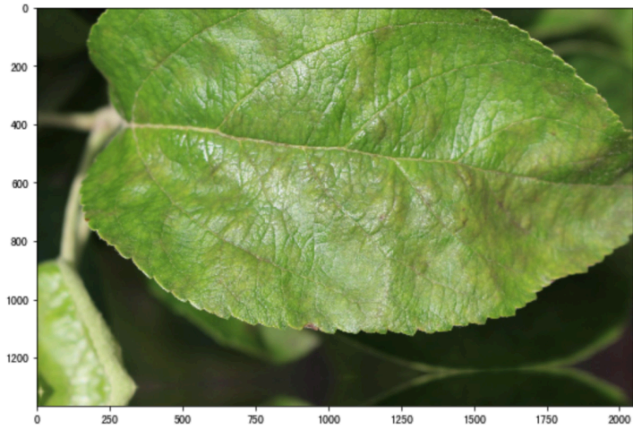
- 上下翻转

- 左右翻转

- 随机缩放

2.2.3、数据集加载

```
import torch  # PyTorch深度学习框架
import torchvision.transforms as transforms  # 图像预处理工具
import torch.nn as nn  # 神经网络模块
from torch.utils.data import Dataset, DataLoader  # 数据集加载工具

# 自定义数据集类（核心组件：数据加载）
class ImageFolderDataset(Dataset):
    def __init__(self, root, label_file, transform=None):
        """
        初始化数据集类（关键步骤：数据路径处理和数据转换设置）
        - root: 图像存储根目录
        - label_file: 包含图像标签的JSON文件路径
        - transform: 图像预处理流水线
        """
        self.root = root
        self.transform = transform
        self.data_paths = []  # 存储图像路径
        self.labels = []      # 存储对应标签

        # 读取标签文件（关键步骤：数据标注加载）
        with open(label_file, 'r') as f:
            lines = f.readlines()
            for line in lines:
                image_name, label = line.strip("\n").split('\t')
                image_path = os.path.join(self.root, image_name)
                self.data_paths.append(image_path)
                self.labels.append(int(label))

    def __len__(self):
        """返回数据集样本数量（必要方法）"""
        return len(self.data_paths)

    def __getitem__(self, idx):
        """获取单个样本（核心方法：数据加载和预处理）"""
        image_path = self.data_paths[idx]
        label = self.labels[idx]
        image = Image.open(image_path).convert('RGB')  # 加载图像并转为RGB

        # 应用数据增强/预处理（关键步骤：图像变换）
        if self.transform:
            image = self.transform(image)
        return image, label

# 数据准备组件（核心组件：数据预处理和加载器创建）
def get_dataset_plant(args):
    # 训练集数据增强策略
    train_transform = transforms.Compose([
        # 1. 缩放控制（指定尺寸，替代原随机裁剪的尺寸固定需求）
        transforms.Resize((256, 256)),  # 先统一缩放到256x256，为后续裁剪留空间
        # 2. 随机裁剪缩放（保留原有功能，裁剪到目标尺寸224x224）
```

```
        transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),  # scale控制裁剪区域占比（0.8-1.0）
        # 3. 随机翻转（包含水平+垂直，即上下左右翻转）
        transforms.RandomHorizontalFlip(p=0.5),  # 随机水平翻转（左右），概率0.5
        transforms.RandomVerticalFlip(p=0.5),    # 随机垂直翻转（上下），概率0.5
        # 4. 随机颜色增强（亮度=光照，对比度）
        transforms.ColorJitter(
            brightness=(0.5, 1.5),  # 随机光照增强：亮度范围0.5-1.5（<1变暗，>1变亮）
            contrast=(0.5, 1.5)     # 随机对比度增强：对比度范围0.5-1.5
        ),
        # 5. 转为张量+标准化（保留原有功能）
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])


    # 验证集预处理（关键区别：无数据增强，仅做中心裁剪）
    val_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])

    # 创建数据集实例（核心对象：训练/测试数据容器）
    train_dataset = ImageFolderDataset(
        root=args.base_path,
        label_file=args.label_file_train,
        transform=train_transform
    )
    val_dataset = ImageFolderDataset(
        root=args.base_path,
        label_file=args.label_file_val,
        transform=val_transform
    )
    test_dataset = ImageFolderDataset(
        root=args.base_path,
        label_file=args.label_file_val,
        transform=val_transform
    )
    # 创建数据加载器（关键组件：批量数据生成器）
    train_loader = DataLoader(
        train_dataset,
        shuffle=True,          # 训练时打乱数据
        batch_size=args.batch_size,
        num_workers=4          # 多进程加速（实际使用时根据硬件调整）
    )
```

```
    val_loader = DataLoader(
        val_dataset,
        shuffle=False,          # 测试时不需打乱
        batch_size=args.batch_size
    )
    test_loader = DataLoader(
        test_dataset,
        shuffle=False,          # 测试时不需打乱
        batch_size=args.batch_size
    )
    return train_loader, val_loader, test_loader
```

2.3、模型

2.3.1、Batch Normalization

> Ref: 1、https://www.bilibili.com/video/BV12d4y1f74C/?spm_id_from=333.788.videopod.sections&vd_source=0c681b71f162c7f69342fa800b58ec82
>
> 2、《鱼书》 6.3小节。
>
> 3、https://www.cnblogs.com/LXP-Never/p/11566064.html#blogTitle0

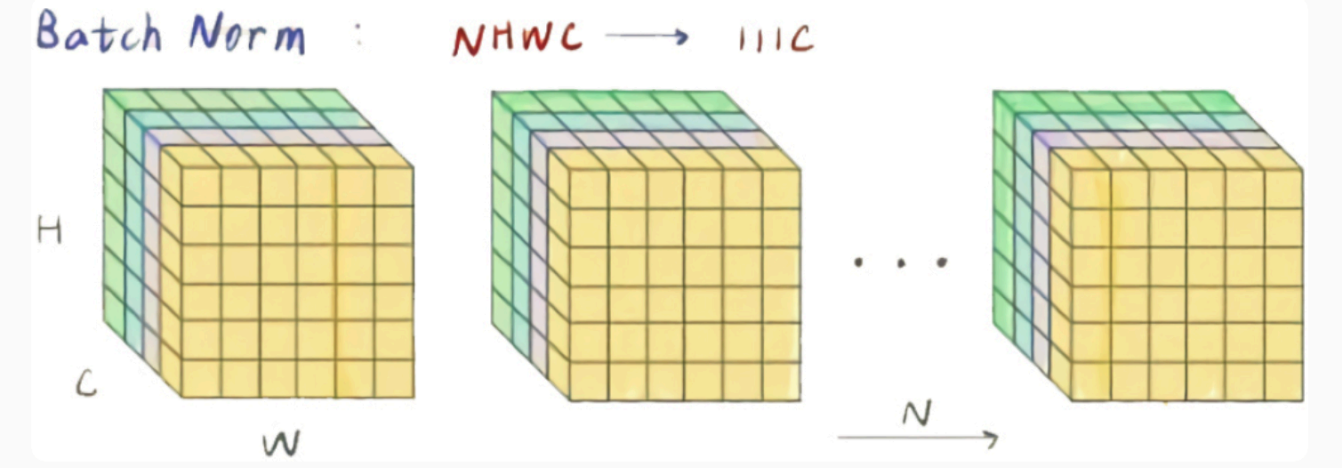
https://arxiv.org/pdf/1502.03167

在训练神经网络过程中，通常输入batch个数据进行训练，这样每个batch具有不同的分布，使模型训练起来相对困难。

批标准化（Batch Normalization, BN）是深度学习中一种重要的技术，通过对神经网络每层输入数据进行归一化（使其均值接近 0，方差接近 1）来稳定和加速模型训练。它能有效解决深层网络中的内部协变量偏移（Internal Covariate Shift, ICS）问题，提高收敛速度，并允许使用更大的学习率，同时还具有一定的正则化效果。

思想

沿着通道维度，在batch维度上，计算(N, H, W)均值和方差，然后对feature map进行归一化，这样不仅数据分布一致，而且避免发生梯度消失。保证每一次数据经过归一化后还保留原有学习来的特征，同时又能完成归一化操作，加速训练。



- **1. 计算均值与方差：**

在一个 mini-batch（小批量）中，对每一维特征计算其均值 μ 和方差 σ^2。

- **2. 归一化：**

将该批次的数据 x_i 转换为：

$$\tilde{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}},$$

使均值变为 0，方差变为 1（其中 ϵ 是一个很小的常数，用于防止除零）。

3. 缩放与平移：

为保留网络的表达能力，引入可学习参数 γ（缩放）和 β（平移），进行线性变换：

$$y_i = \gamma \tilde{x}_i + \beta,$$

得到最终输出 y_i。

BN 的测试时候使用

> 执行model.eval()，BN层中的参数会自动锁定。

训练与测试阶段的不一致问题：Batch Normalization 的一个潜在问题是： 在训练阶段通常使用较大的 batch size，而在测试（推理）阶段 batch size 往往为 1。然而，BN 的均值和方差依赖于 batch 统计量，这会导致训练与测试阶段的行为不一致。

为解决该问题，BN 在训练过程中使用**移动平均（moving average）** 来估计全局均值与方差，并在推理阶段使用这些全局统计量，而不是某个特定 mini-batch 的统计量。这样可以保证模型在测试阶段的输出稳定性与一致性。-

- 移动平均的计算方式

移动平均通过如下公式更新全局均值与方差：

$$\mu_{\text{global}} \leftarrow (1 - \text{momentum}) \cdot \mu_{\text{global}} + \text{momentum} \cdot \mu_{\text{batch}},$$

$$\sigma^2_{\text{global}} \leftarrow (1 - \text{momentum}) \cdot \sigma^2_{\text{global}} + \text{momentum} \cdot \sigma^2_{\text{batch}},$$

其中：

- $\mu_{\text{batch}}, \sigma^2_{\text{batch}}$：当前 mini-batch 的均值与方差；
- $\mu_{\text{global}}, \sigma^2_{\text{global}}$：移动平均得到的全局统计量；
- **momentum**：动量参数，通常取值在 0.01 到 0.1 之间，用于控制统计量更新速度。

BN 的使用位置

BN 通常放置在**激活函数之前**，常见结构包括：

- **ResNet V1：** $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$
- **ResNet V2：** $\text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}$

无论采用哪种结构形式，BN 都始终位于非线性激活函数之前，用于对线性变换后的特征进行归一化处理。

主要优点

- **加速收敛：** 将输入分布稳定在非饱和区，缓解梯度消失与梯度爆炸问题，使训练更快。
- **提高稳定性：** 减少层与层之间的依赖，允许使用更大的学习率。
- **降低对初始化的敏感度：** 不再过度依赖精细的权重初始化策略。
- **正则化效果：** Mini-batch 统计量引入的噪声具有正则化作用，可减少对 Dropout 的依赖。
- **支持传统激活函数：** 允许在深层网络中安全使用 Sigmoid、Tanh 等激活函数。

Torch

- https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html

```
# With Learnable Parameters
m = nn.BatchNorm2d(100)
input = torch.randn(20, 100, 35, 45)
output = m(input)
```

2.3.2、自定义模型

- 使用1.10中自定义的模型，调整超参数看看。

2.3.3、ResNet

- 使用现成的网络，ResNet, paper: https://arxiv.org/pdf/1512.03385

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

- ResNet文章摘要

```
Deeper neural networks are more difficult to train.
更深的神经网络更难训练。
We present a residual learning framework to ease the training of networks that are
substantially deeper than those used previously.
我们提出了一种残差学习框架，用于缓解训练比以往网络深得多的模型所面临的困难。
We explicitly reformulate the layers as learning residual functions with reference to the
layer inputs, instead of learning unreferenced functions.
我们明确地将各层重新表述为学习相对于层输入的残差函数，而不是学习与输入无关的函数。
```

We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth.
我们提供了全面的实验证据，表明这些残差网络更容易优化，并且可以从显著增加的深度中获得更高的准确率。
On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [41] but still having lower complexity.
在 ImageNet 数据集上，我们评估了深度高达 152 层的残差网络——比 VGG 网络 [41] 深 8 倍，但复杂度仍然更低。
An ensemble of these residual nets achieves 3.57% error on the ImageNet test set.
这些残差网络的集成模型在 ImageNet 测试集上取得了 3.57% 的错误率。
This result won the 1st place on the ILSVRC 2015 classification task.
该结果赢得了 ILSVRC 2015 分类任务的第一名。
We also present analysis on CIFAR-10 with 100 and 1000 layers.
我们还在 CIFAR-10 数据集上对 100 层和 1000 层的网络进行了分析。
The depth of representations is of central importance for many visual recognition tasks.
表示的深度对于许多视觉识别任务至关重要。
Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset.
仅仅由于我们极深的表示，在 COCO 目标检测数据集上就获得了 28% 的相对提升。
Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.
深度残差网络是我们参加 ILSVRC 和 COCO 2015 竞赛的基础，我们也在 ImageNet 检测、ImageNet 定位、COCO 检测以及 COCO 分割等任务中获得了第一名。

2.4、损失函数及优化器

- 损失函数和输出层设计
    - 对应之前讲过的深度学习入门：基于 python 的理论和实现中的4.4节和5.2.2小节。

```
# 损失函数和优化器（核心组件：学习目标定义和优化策略）
criterion = nn.CrossEntropyLoss()  # 分类任务常用损失函数
optimizer = optim.SGD(  # 随机梯度下降优化器
    model.parameters(),
    lr=args.learning_rate,
    momentum=0.9  # 可添加动量参数（常见优化技巧）
)
```

2.5、模型训练

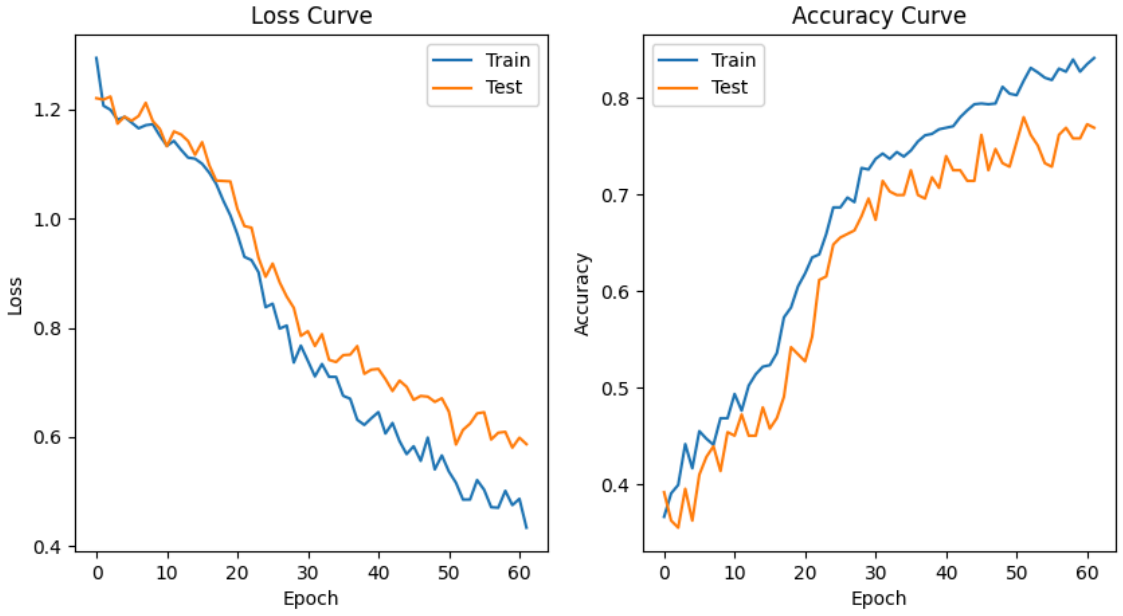
```
# 训练循环（核心流程：迭代优化）
for epoch in range(args.epochs):
    # 单个epoch训练
    train_loss, train_acc = train(
        model, train_loader, criterion,
        optimizer, epoch, device
    )

    # 模型验证
    val_loss, val_acc = test(
        model, valtest_loader, criterion, device
    )
```

2.6、完整代码

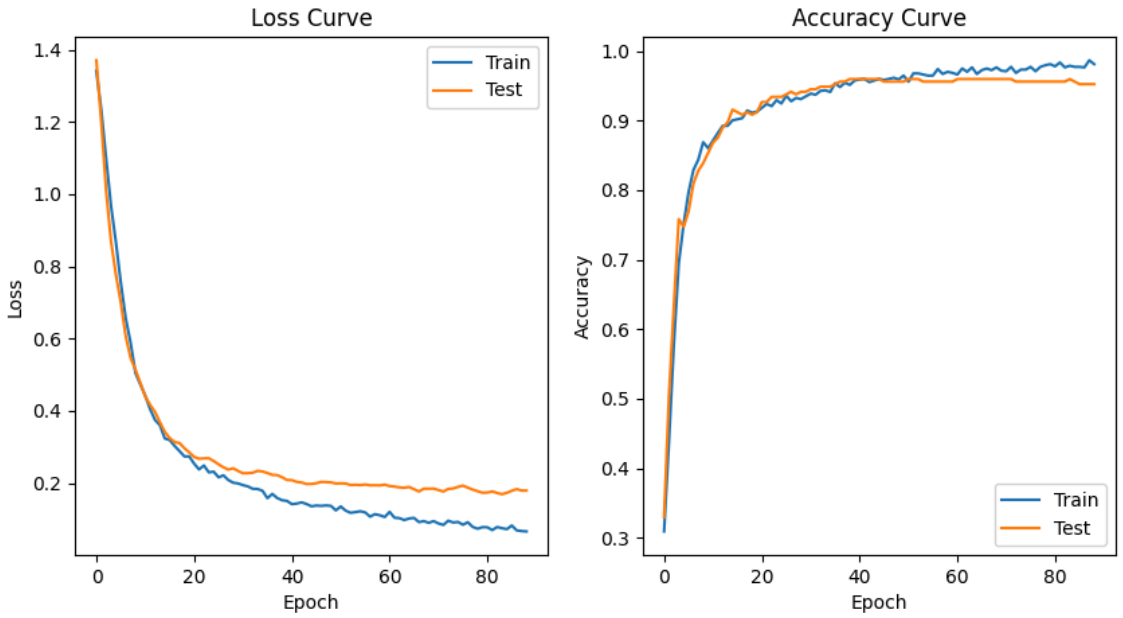
- 代码地址：链接: https://pan.baidu.com/s/1BZIoYkatVT1NrI8MFBkmag?pwd=6vcr 提取码: 6vcr
- 上传autodl 路径，能够保证每次开机在同一个区，不同机器都能访问到。

2.7、超参数调整




```
python3 train.py  --model_name resnet18  --load_pretrained --batch_size 256  --learning_rate
0.001 --epochs 300 --save_fig_name resnet18_256_1e3_300

Test Loss: 0.216, Accuracy: 95.97%
test_loss: 0.2161743864417076 test_acc: 0.9597069597069597
```



三、优化及更换数据集

3.1、优化

3.1.1 模型融合

加权融合通过对多个模型的预测概率进行加权平均，权重根据验证集性能分配，使整体预测结果更加稳定和准确。

（1）加权融合实现方式

假设我们有 **3 个模型**（Model A、B、C），对同一张图片的预测概率如下（4 类任务）：

- Model A 输出：`[0.10, 0.60, 0.20, 0.10]`
- Model B 输出：`[0.20, 0.50, 0.20, 0.10]`
- Model C 输出：`[0.05, 0.70, 0.15, 0.10]`

在验证集上的准确率分别为：

- Model A：90%
- Model B：85%
- Model C：95%

（2）权重设定方式

根据验证集准确率分配权重，并进行归一化：

$$w_A = \frac{0.90}{0.90 + 0.85 + 0.95}, \quadw_B = \frac{0.85}{0.90 + 0.85 + 0.95}, \quadw_C = \frac{0.95}{0.90 + 0.85 + 0.95}$$

计算后约为：

- (w_A = 0.33)

- (w_B = 0.31)
- (w_C = 0.36)

（3）加权求和

对每一类的概率进行加权求和：

- 类别 0：
  (0.10×0.33 + 0.20×0.31 + 0.05×0.36 ≈ 0.11)
- 类别 1：
  (0.60×0.33 + 0.50×0.31 + 0.70×0.36 ≈ 0.60)
- 类别 2：
  (0.20×0.33 + 0.20×0.31 + 0.15×0.36 ≈ 0.18)
- 类别 3：
  (0.10×0.33 + 0.10×0.31 + 0.10×0.36 ≈ 0.10)

（4）最终预测

加权后的概率最大的是 **类别 1**，因此最终预测类别为 **1**。

3.2、更换数据集

> 修改好的数据集和代码见：百度网盘

3.2.1、数据集介绍

- 花卉识别，数据集下载地址：flower-image-dataset，更多数据集可以去kaggle等网站上找：kaggle
- 共有10种不同的花卉，分别是：郁金香、兰花、牡丹、绣球花、百合花、栀子花、花园玫瑰、雏菊、木槿和三角梅。

Total: 733

Train: 513

Val ：109

Test : 111

3.2.2、更换流程

- 上传数据集、产生新数据的label文件
- 更换模型类别数
- 详细见视频。

四、简历怎么写?

可能和视频讲的稍微不符合，以最新文档为准。

复制地址：项目2-可下载

基于卷积神经网络的苹果叶片病害图像分类（CVPR挑战赛）

任务描述：开发用于苹果叶片病害识别的深度学习模型，实现对健康叶片及三类病害的分类。

实现方法：

1. **数据收集与预处理**
    - 使用 CVPR FGVC7 植物病理学挑战赛数据集，包含约 1800 张叶片图像，4 个类别；
    - 数据预处理：采用标准化、随机裁剪、翻转及颜色抖动等数据增强方法，提升数据多样性与模型泛化能力。
2. **模型构建**
    - 尝试设计 CNN 网络结构，包含卷积层、激活函数、池化层及残差连接，用于提取图像特征；
    - 使用 ResNet 等经典网络架构作为对比模型，以进一步提升分类性能。
3. **模型训练**
    - 基于 PyTorch 框架实现训练流程，采用交叉熵损失函数作为优化目标；
    - 对 SGD、带动量SGD、AdaGrad和Adam 等优化器进行实验比较，并超参数调优（如 epochs、batch size），不断优化模型效果；
    - 通过对比验证集性能，多次调整网络结构与参数配置，确定最优模型方案。

算法优化：

1. **缓解过拟合问题：**使用 L2 正则化、Dropout、数据增强及早停；
2. **应对数据量较少问题：**引入迁移学习方法；
3. **为进一步增加模型的泛化性：**使用多模型融合，将resnet18、resnet34和自定义的模型加权融合。

取得结果：项目在测试集上达到 79.2% 的准确率。

注意事项：

- 根据你实际掌握情况，选择性的写（使用 L2 正则化、Dropout 技术如果不懂就不要写。）

- 大体的写的逻辑是： 1、任务描述；2、实现方法、3、解决难点；4、取得结果。

基于卷积神经网络的苹果叶片病害图像分类（CVPR挑战赛）

任务描述：开发用于苹果叶片病害识别的深度学习模型，实现对健康叶片及三类病害的分类。

实现方法：1.数据收集与预处理：使用 CVPR FGVC7 植物病理学挑战赛数据集并进行数据预处理。2.**模型构建:**尝试设计 CNN 网络结构和使用 ResNet 等经典网络结构。3.**模型训练：**基于 PyTorch 框架实现训练流程，采用交叉熵损失函数作为优化目标；对 SGD、带动量SGD、AdaGrad和Adam 等优化器进行实验比较，并超参数调优（如 epochs、batch size），不断优化模型效果；

取得结果：在测试集上达到 79.2% 的准确率。

上面写的是基础的版本，如果掌握了解决过拟合、模型融合，可以加入到简历中（推荐报考211及以上的加入）。

五、面经

0、背景知识

1、简单介绍一下这个项目。

本项目是CVPR 细粒度视觉分类植物病理学挑战赛，该挑战赛的目标是训练一个图片分类模型来识别苹果叶片的不同病害类型及健康状态。项目采用 PyTorch 深度学习框架构建卷积神经网络模型，通过数据增强、迁移学习、Dropout、L2正则化、尝试不同超参数等技术提升模型性能，实现在测试集上达到了79.2 Top-1 Acc。

2、该任务属于哪一类机器学习问题？为什么？

该任务属于监督学习中的分类问题。因为每个样本都有明确的类别标签，模型需要学习从输入图像到离散类别标签之间的映射关系。

3、项目中代码量大约是多少？

项目代码总量约为 700～900 行左右，主要包括数据划分脚本、自定义数据集类、数据增强流程、模型构建、训练与验证循环、测试评估函数等模块。

一、数据

1、项目中使用了什么数据集？数据来源与规模如何？

本项目使用的是 CVPR FGVC7 植物病理学挑战赛数据集，可通过 Kaggle 下载。数据集规模如下：

- 训练集：1274 张
- 验证集：273 张
- 测试集：274 张

共计约 1821 张苹果叶片图像，包含 4 个类别。

2、数据特征包含哪些内容？是否进行了预处理？

每个样本为 RGB 彩色图像，包含叶片的颜色、纹理和病斑等视觉特征。在建模前对数据进行了如下预处理与增强：

- 随机裁剪与缩放：增强模型对目标尺度变化的适应能力；
- 随机水平与垂直翻转：模拟不同拍摄角度；
- 随机亮度与对比度扰动：增强模型对光照变化的鲁棒性；
- 标准化处理：使输入分布与预训练模型保持一致，加速收敛。

3、标签的类别与分布情况如何？

标签共包含 4 个类别：

a）healthy（健康叶片）：516

b）rust（雪松苹果锈病）：622

c）scab（苹果黑星病）：592

d）multiple_diseases（多种病害共存）：91

各类别样本数量存在一定不均衡，其中多病害类别样本较少，整体呈现轻度类别不平衡特征。

4、图像的原始大小是多少？输入模型的大小resize是多少？为什么要对输入的图像都 resize 到 224？

原始的2048×1365，输入模型的大小为 224×224 ，将图像 resize 或裁剪到 224×224 是为了与预训练模型（如 ResNet 在 ImageNet 上训练时的输入尺寸）保持一致，使模型能够直接使用预训练权重。

二、网络结构

1、介绍一下 ResNet 中的残差结构？

随着网络不断加深，训练过程会面临梯度消失等问题。ResNet 通过引入**残差连接（skip connection）**，有效缓解了这一困难。

ResNet 的核心思想是让网络学习残差映射，即：

$$\mathcal{H}(x) = F(x) + x$$

这种结构使得信息和梯度可以在网络中直接传播，从而支持构建非常深的网络结构。

2、输入图像是 (3, 224, 224)，经过

nn.Conv2d(in_channels=3, out_channels=2, kernel_size=3, stride=2, padding=0, bias=True)
之后输出的维度是什么？conv2d 中有几个卷积核？每个卷积核的可学习参数是多少？

$$\text{输出尺寸} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

代入参数：$\frac{224 - 3 + 0}{2} + 1 = 111$，因此输出尺寸为 (2, 111, 111)。

该卷积层共有 2 个卷积核（因为 out_channels=2）。每个卷积核的参数数量为：

- 权重参数：3（输入通道数） × 3 × 3 = 27
- 偏置参数：1
  总计每个卷积核 28 个参数，两个卷积核共 56 个参数。

3、什么是 Batch Normalization？它在模型训练中起什么作用？

Batch Normalization 是一种对中间特征进行标准化的方法，通过对每个 mini-batch 的特征进行均值为 0、方差为 1 的归一化，并引入可学习的缩放和平移参数，使网络训练更加稳定。

其作用包括：加速收敛、缓解梯度消失与爆炸问题、允许使用更大的学习率，并具有一定的正则化效果。

4、你都知道哪些做图像分类的神经网络？

常见的图像分类网络包括：

- 基于卷积的：LeNet、AlexNet、VGG、GoogLeNet（Inception）、ResNet等。
- 基于Transformer的：Vision Transformer（项目5会讲）

5、我们直接加载 ResNet 原始的网络权重之后，需要修改什么才能使用？

需要将最后的全连接层（分类层）替换为输出维度为 4 的新全连接层，以适应本任务的类别数量。

三、损失函数

1、本项目使用了什么损失函数？

项目使用交叉熵损失函数（CrossEntropyLoss）。更具体的见深度学习基础文档里面。

四、优化器

1、项目中使用了哪些优化器？哪一个效果更好？为什么？

项目中尝试了多种优化器，包括：SGD、带动量的 SGD、AdaGrad 以及 Adam。综合实验结果来看，AdamW表现最好，其在收敛速度、稳定性和最终性能方面均优于其他方法。

为什么的话，就介绍这四种优化器的区别，见深度学习基础文档。

五、训练超参数

1、项目中调节了哪些训练超参数?

主要调节的训练超参数包括：学习率（learning rate）、批大小（batch size）、训练轮数（epochs）、动量系数（momentum）以及权重衰减系数（L2 正则化）。

2、效果最好的超参数是什么？在什么显卡训练的，训练了多久?

- 模型：ResNet18（加载预训练权重，4 类输出）
- 损失函数：CrossEntropyLoss
- 优化器：Adam（lr=0.001，weight_decay=1e-4）
- 批大小：32
- 最大训练轮数：300
- 早停机制：验证集准确率连续 50 个 epoch 无提升即停止

训练情况：

- 最优结果出现在第 70 个 epoch
- 使用显卡：RTX 4090
- 训练时间：约 1 小时

六、评价指标

1、本项目使用什么评价指标评估模型性能?

项目主要使用 Top-1 Accuracy 作为评价指标，即模型预测概率最高的类别是否与真实类别一致。

七、训练结果与优化

1、项目中遇到的主要问题是什么？是如何判断的？

主要问题是过拟合。训练集与验证集之间存在明显的性能差距,训练集上的损失函数持续下降、准确率持续上升，但验证集和测试集top1-acc在达到60左右后不再提升甚至下降，表现出明显的“训练好、泛化差”的特征。

2、为缓解过拟合采取了哪些优化措施？这些措施产生了什么影响？

- 更换小一些模型，比如从resnet34更换为resnet18
- 数据增强，提高样本多样性；
- 使用迁移学习。
- Dropout、L2正则化、早停
- 调整超参数
- 多模型融合

3、什么是迁移学习？它在本项目中如何发挥作用？

迁移学习是指利用在大规模数据集（如 ImageNet）上训练得到的模型参数作为初始化权重，并在目标任务上进行微调。

在本项目中，通过加载预训练的 ResNet 模型并替换最后分类层，使模型在小样本数据集上能够更快收敛并获得更好的分类性能。

4、如果进一步改进该项目，可以从哪些方面入手？

在模型层面，可以尝试更深的网络结构，如 Vision Transformer（项目5会讲）；

在训练策略层面，可以引入学习率调度；

同时可以针对长尾类别问题进行数据平衡处理，或尝试模型融合方法进一步提升性能。

5、在尝试模型融合方法时，加权融合的具体实现方式是怎样的？如何确定加权融合中的权重？

加权融合是指对多个模型输出的类别概率按权重加权求和，然后选择概率最大的类别作为最终预测结果。

权重通常根据各模型在验证集上的性能指标（如准确率或损失）进行设定，性能更好的模型分配更高权重。